اونيۏرسيتي مليسيا قهڠ السلطان عبد الله
UNIVERSITI MALAYSIA PAHANG
AL-SULTAN ABDULLAH

Centre for
Mathematical Sciences
Pusat Sains Matematik

BSD3523:
MACHINE LEARNING

GROUP PROJECT REPORT
(Data-Driven Solutions for Targeted Poverty Relief)

EmpowerAid Initiatives
Authored by: Almira Damia Binti Syahnizam (SD22002)
Nurkhairul Izzati Binti Mohd Sallehan (SD22005)
Vienosha A/P Selvaraju (SD22065)
Nur Atieka Rafiekah Binti Razak (SD22003)

TABLE OF CONTENTS

1.0 INTRODUCTION

1.1 Overview of the Company

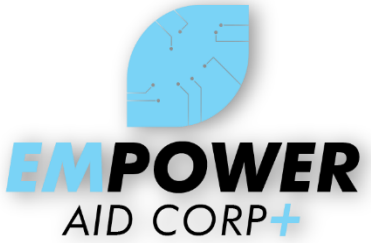


Figure 1.1 Company logo

EmpowerAid Initiative is a consulting start-up company that focuses on using data driven to solve any socioeconomic issues. We are experts in data analytics and machine learning to solve complex problems that challenge the community or organization. Our mission is to educate the communities by integrating machine learning to provide better decision-making. For example, one of our latest projects is to help identify the regions most affected by poverty using machine learning algorithms. We analyze the large dataset that consists of information related to economics, demographics and education where this significant insight can help governments and community organizations highlight some targeted areas that need help. This approach enhances the decision making can also reduce cost and time in reducing poverty in Malaysia.

1.2 Role of Each Group Member

Our team at EmpowerAid Initiative are all experts in data analysis that are dedicated to transforming data into insightful information. Other than that, we also have a diverse set of skills and expertise that ensure every aspect of the project is covered to deliver effective solutions.

Table 1.2.1 Role of member in the company

Role	Name	Responsibilities
Chief Executive Officer (CEO)	Almira Damia Binti Syahnizam	• Lead the team and ensure everyone knows their role • Make final decision on the discussion related to the project.
Chief Technology Officer (CTO)	Nur Atieka Rafiekah Binti Razak	• Decide on the technology used such as selection and

		implementation of tools, technologies, and methodologies. • Guide step if there are any challenges with data or tools used.
Machine Learning Engineer	Nurkhairul Izzati Binti Mohd Sallehan	• Assist in building the model using the machine-learning method • Check the model whether it is working well or needs improvement to achieve better accuracy.
Data Analyst	Vienosha A/P Selvaraju	• Responsible for guiding and analyzing the data that have been collected. • Explain and interpret the progress in non-technical terms to the client.

2.0 PROJECT OVERVIEW

2.1 Project Description

Poverty relief is about helping people who don't have enough money or resources to live a decent life. It includes giving short-term help, like food and shelter, also creating long-term solutions, like education and job training. Understanding poverty relief is important because poverty doesn't just affect individuals but also affects the whole society. It slows down progress, increases inequality and can cause social problems. By fighting poverty, we can help people reach their full potential, make communities more inclusive and build a better future. Learning about poverty relief helps us understand why poverty happens, see what works to fix it and take action to make the world fairer for everyone.

In this project, we will discuss more on solution for poverty in aspects of education, household and employment in Malaysia. The dataset use in this project are taken from various sources such as Department of Statistics Malaysia website and data.gov. This project will be carried out by utilizing Python language and SQL queries for doing all the machine learning processes. The main goal of this project is to develop a predictive modelling to see what influences poverty to occur.

In addition to exploring solutions in education, household and employment, this project aims to identify patterns and trends that contribute to poverty in Malaysia. By analysing the data, we hope to uncover the root causes of poverty, such as regional disparities, access to resources and economic instability. The predictive model will not only provide insights into current poverty dynamics but also offer actionable recommendations for policymakers and NGOs to prioritize interventions effectively. By leveraging machine learning techniques, the project will contribute to data-driven solutions that can help break the cycle of poverty, ensuring that resources are allocated where they are most needed to create sustainable and inclusive growth.

2.2 Problem Statement

In Malaysia, poverty remains to be one of the complex issues where it affects both the rural and urban areas with significant regional disparities. Even though the country has achieved a notable growth of economics, this issue continues to challenge the socio-economic development. These disparities disproportionately impact communities especially in the states like Sabah and Kelantan, where the access to resources and living facilities remains limited (Md Shah et al., 2023).

Poverty alleviation programs are currently undergoing some challenges that related to the implementation, monitoring and inconsistent definition of poverty. According to Nor & Khelghat-Doost (2019), programs like 1AZAM have underscored the inefficiencies in resource allocation and gaps in addressing these poverty's root causes as for example. Additionally, the demographic and rising cost of living differences continue to worsen the socioeconomic weakness across the country (Rahman et al., 2021).

Therefore, there still exist gaps in understanding what causes poverty and the factors sustaining inequality. These gaps hinder efforts to achieve more equal and efficient socio-economic development in Malaysia. The lack of effective and evidence-based solutions to address these gaps further hinders efforts to provide sustainable poverty relief in Malaysia.

2.3 Objectives

The objectives of this study are as follows:

1. To analyze the regional disparities of poverty in Malaysia through exploratory data analysis.
2. To develop a predictive model to understand the factors influencing poverty.
3. To identify the key factors of poverty using feature importance derived from the predictive model.
4. To propose actionable solutions for overcoming poverty by leveraging insights from the predictive model.

2.4 Scope of the Project

This research will investigate the regional poverty disparities in Malaysia with the help of easily accessible data from national sources. The objective of the study is to create a prediction model by finding the key influences of poverty. Statistical analysis and machine learning techniques will be utilized through Python, mostly using libraries like Pandas, Scikit-learn and Matplotlib. The study covers a dataset of the last four years which is from 2019 - 2022, which guarantees that the issue is up to date. The proposed solutions will be the ones raised by the best performing predictive model and will be in line with the Malaysian socio-economic environment.

2.5 Limitations of the Project

The study relies on available and quality secondary data that might have missing values or inconsistencies and as a result, not all regions are well represented. Access to data that is detailed and confidential is restricted, which in turn may affect the granularity of the analysis. Furthermore, the findings of the predictive model may not be universally relevant for all the regions due to their distinctive socio-economic conditions and thus the solutions that were proposed may be confronted with obstacles including policy resistance and resource constraints. Although these limitations exist, the study should be able to provide a significant number of insights into poverty trends and potential solutions.

3.0 DATA PREPARATION

3.1 Data Description

In this project, nine datasets are used to fulfill the objective and complete our study. The dataset consists of six tables. The description of all datasets will be described in detail as provided in Table 3.1.1 – 3.1.6.

Table 3.1.1 Data Description of Poverty by Administrative District

Variable	Data Type	Description
state	String	Represents 16 states in Malaysia, including 3 Federal Territories
district	String	Represents 160 administrative districts from the states, including 4 states (Perlis and the 3 Federal Territories which do not have subdivisions)
date	Date	The date in YYYY-MM-DD format, with MM-DD set to 01-01 as the data is annual frequency
poverty_absolute	Float	Proportion of households with monthly income below the Poverty Line Income (PLI)
poverty_relative	Float	Proportion of households with monthly income below half the district median income

Table 3.1.2 Data Description of Household Income and Expenditure by Administrative District

Variable	Data Type	Description
state	String	One of 16 states, including the 3 Federal Territories
district	String	One of 160 administrative districts, including the 4 states (Perlis and the 3 Federal Territories which do not have subdivisions
date	Date	The date in YYYY-MM-DD format, with MM-DD set to 01-01 as the data is at annual frequency
income_mean	Integer	Average gross monthly household income
income_median	Integer	Median gross monthly household income, this is generally less than the mean income due to the right-skewed distribution of income

Table 3.1.3 Data Description of Annual Labor Force by Districts

Variable	Data Type	Description
state	String	One of 16 states
district	String	One of 160 administrative districts. However, it should be noted that only 140 administrative districts are represented in this dataset due to constraints on data availability at district level for more remote areas
date	Date	The date in YYYY-MM-DD format, with MM-DD set to 01-01 since the data is at annual frequency
lf	Float, '000	The number of employed and unemployed individuals. This figure also represents the number of people participating in the labour force.
lf_employed	Float, '000	The number of people who worked at least one hour for pay, profit or family gain, in thousands of people
lf_unemployed	Float, '000	The number of people who did not work but were looking for work or available to work
lf_outside	Float, '000	The number of people not classified as employed or unemployed, including housewives, students, early retired, disabled persons and those not interested in looking for a job
p_rate	Float	Ratio of unemployed to labour force size
u_rate	Float	Ratio of the labour force size to the working-age (15-64 population)
ep_rate	Float	Ratio of the number of employed people to the working-age (15-64 population)

Table 3.1.4 Data Description of Education Completion Rate by State

Variable	Data Type	Description
date	Date	Date in YYYY-MM-DD format, with MM-DD set to 01-01 as the data is at annual frequency
state	String	One of 16 states, or Malaysia for national-level data
stage	String	Either primary (Standards 1 to 6, lower secondary (Form 1 to 3), or upper secondary (Form 4 to 5)
sex	String	Either both sexes, male, or female
completion	Float	Proportion of students who complete that stage of education, relative to the total cohort that started that stage

Table 3.1.5 Data Description of Education Enrollment by District

Variable	Data Type	Description
date	Date	Date in YYYY-MM-DD format, with MM-DD set to 01-01 as the data is at annual frequency
state	String	One of 16 states, or Malaysia for national-level data
district	String	One of the administrative districts, or 'All Districts' for state-level data
stage	String	Either primary (Standards 1 to 6, secondary (Form 1 to 5), or post-secondary education (Form 6)
sex	String	Either both sexes, male, or female
students	Integer	Number of students enrolled

Table 3.1.6 Data Description of CPI Inflation Dataset

Variable	Data Type	Description
state	String	Represents one of 16 states in Malaysia (eg: Kelantan, Selangor)
date	Date	The date of observation in YYYY-MM-DD format. The DD field is set to 01 as the data reflects the monthly frequency
division	String	A 2-digit code from 01-13 or 'overall', indicates specific divisions within a state. The division codes matched to categories in the MCOICOP to a Lookup table, which provides definitions in English and Malay.
inflation_yoy	Float	The year-on-year inflation rate is calculated as the percentage change in the CPI compared to the previous year.
inflation_mom	Float	Month-on-month inflation rate, calculated as the percentage change in the CPI compared to the previous month 1 month ago

3.2 Data Preprocessing

3.2.1 Before Combing Dataset

i) Import dataset

```
data2 = pd.read_csv('poverty_district.csv')
data2.head()
```

	state	district	date	poverty_absolute	poverty_relative
0	Johor	Batu Pahat	1/1/2019	2.9	9.0
1	Johor	Batu Pahat	1/1/2022	5.1	19.4
2	Johor	Johor Bahru	1/1/2019	3.3	12.8
3	Johor	Johor Bahru	1/1/2022	3.7	10.4
4	Johor	Kluang	1/1/2019	5.0	24.9

Figure 3.1 Import dataset

The dataset is loaded into a dataframe using the read_csv function in python. The head() method is then used to preview the first few rows of the dataset, which provides an overview of its structure and content.

ii) Info of dataset

```
print(data2.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 318 entries, 0 to 317
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   state             318 non-null    object
 1   district          318 non-null    object
 2   date              318 non-null    object
 3   poverty_absolute  318 non-null    float64
 4   poverty_relative  318 non-null    float64
dtypes: float64(2), object(3)
memory usage: 12.5+ KB
None
```

Figure 3.2 Info of the dataset

The .info() function is used to view the data type of each attribute in the dataset, also allowing to know the number of entries and columns. This step is to understand the characteristics of each of the features.

iii) Checking missing and duplicate values

```
# CHECKING MISSING VALUES
print(data2.isnull().sum())
```

```
state               0
district            0
date                0
poverty_absolute    0
poverty_relative    0
dtype: int64
```

```
# CHECKING DUPLICATED VALUES
data2.duplicated().sum()
```

```
0
```

Figure 3.3 Checking missing and duplicate values

In this step, the dataset is checked for missing and duplicate values to ensure the data integrity. The isnull().sum() function is used to calculate the presence of missing values in each column. Similarly, the duplicated().sum() function is used to check the duplicate rows. Checking and handling the values in this step ensures the completeness and uniqueness of the dataset before proceeding to further analysis.

iv) Formatting the features

```
data2['date'] = pd.to_datetime(data2['date'])
```

```
data2['year'] = data2['date'].dt.year
```

```
data2 = data2[(data2['year'] >= 2019) & (data2['year'] <= 2022)]
```

```
data2 = data2.drop(columns=['date'])
```

```
print(data2.head())
```

```
   state     district  poverty_absolute  poverty_relative  year
0  Johor   Batu Pahat               2.9               9.0  2019
1  Johor   Batu Pahat               5.1              19.4  2022
2  Johor  Johor Bahru               3.3              12.8  2019
3  Johor  Johor Bahru               3.7              10.4  2022
4  Johor       Kluang               5.0              24.9  2019
```

Figure 3.4 Formatting the features

In this step, the date column is converted to a datetime format using the pd.to_datetime() to ensure proper handling of the date values. A new column, year is extracted from the date column to simply the analysis and filtering by year. Then the original date column is dropped to avoid redundancy and leaving only necessary columns.

v) Checking and handling the outliers

```
plt.figure(figsize=(8, 6))
sns.boxplot(x=data2['poverty_absolute'])
plt.title('Boxplot Before Cleaning (poverty_absolute)')
plt.xlabel('poverty_absolute ')
plt.show()
```

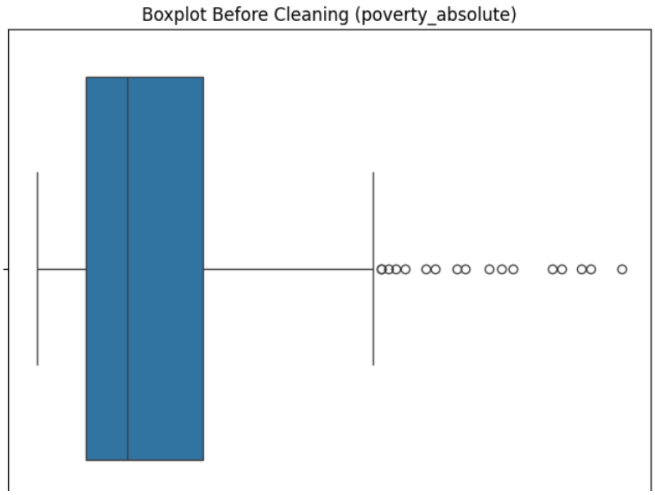


Figure 3.5 Boxplot checking for outliers

```
Q1 = data2['poverty_absolute'].quantile(0.25)
Q3 = data2['poverty_absolute'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

clean_poverty_absolute = data2[(data2['poverty_absolute'] >= lower_bound) & (data2['poverty_absolute'] <= upper_bound)]
```

Figure 3.6 IQR method to handle outliers

```
plt.figure(figsize=(8, 6))
sns.boxplot(x=clean_poverty_absolute['poverty_absolute'])
plt.title('Boxplot After Removing Outliers (poverty_absolute)')
plt.xlabel('poverty_absolute')
plt.show()
```

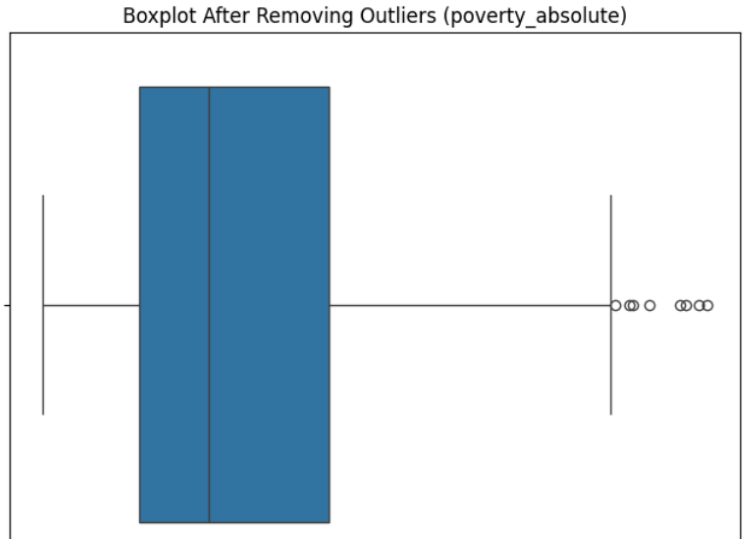


Figure 3.7 Boxplot after handling outliers

Figure 3.2.5 to Figure 3.2.7 illustrates the process of identifying and removing outliers from the numerical features only using the Interquartile Range (IQR) method. The IQR is calculated by subtracting the 25th percentile (Q1) from the 75th percentile (Q3), and outliers are identified as values below the lower bound (Q1 - 1.5 * IQR) or above the upper bound (Q3 + 1.5 * IQR).

3.2.2 Combining the Dataset

After cleaning the dataset for each of the csv files, the data is combined through the application of PostgreSQL. The steps that occurred can be viewed from the figures shown in this subtopic from creating the table of each dataset to saving the final combined data.

i) Creating table for each dataset

```
CREATE TABLE hh_income_district (
    state TEXT,
    district TEXT,
    income_mean NUMERIC,
    income_median NUMERIC,
    year INT
);
```

Figure 3.8 Table for household

```
CREATE TABLE lfs (
    state TEXT,
    district TEXT,
    lf NUMERIC,
    lf_employed NUMERIC,
    lf_unemployed NUMERIC,
    lf_outside NUMERIC,
    p_rate NUMERIC,
    u_rate NUMERIC,
    ep_ratio NUMERIC,
    year INT
);
```

Figure 3.9 Table for labor force

```
CREATE TABLE poverty_district (
    state TEXT,
    district TEXT,
    poverty_absolute NUMERIC,
    poverty_relative NUMERIC,
    year INT
);
```

Figure 3.10 Table for poverty

```
CREATE TABLE enrollment_school_district (
    state TEXT,
    district TEXT,
    stage TEXT,
    sex TEXT,
    students INT,
    year INT
);
```

Figure 3.11 Table for enrollment

```
CREATE TABLE completion_school_state (
    state TEXT,
    stage TEXT,
    sex TEXT,
    completion NUMERIC,
    year INT
);
```

Figure 3.12 Table for completion

```
CREATE TABLE cpi_inflation (
    state TEXT,
    division TEXT,
    inflation_yoy NUMERIC,
    inflation_mom NUMERIC,
    year INT
);
```

Figure 3.13 Table for cpi inflation

The SQL script provided above shows the set of tables designed to structure and organize various datasets required for the analysis in this project. These tables facilitate the process of combining datasets by aligning data at common levels of aggregation such as state, district and year. The key tables include hh_income_district for the household metris, lfs for labor force data, poverty_district for the poverty measures, enrollment_school_district and completion_school_state for education statistics and cpi_inflation. By structuring the data in this way, the tables enable seamless integration and merging of datasets where it promotes efficient analysis.

ii) Uploading the csv file

```
COPY hh_income_district FROM 'C:\Program Files\PostgreSQL\cleaned hh income district.csv' CSV HEADER;
COPY lfs FROM 'C:\Program Files\PostgreSQL\cleaned lfs.csv' CSV HEADER;
COPY poverty_district FROM 'C:\Program Files\PostgreSQL\cleaned_poverty by district.csv' CSV HEADER;
COPY enrollment_school_district FROM 'C:\Program Files\PostgreSQL\enrollment_school_district_cleaned.csv' CSV HEADE
COPY completion_school_state FROM 'C:\Program Files\PostgreSQL\completion_school_state_cleaned.csv' CSV HEADER;
COPY cpi_inflation FROM 'C:\Program Files\PostgreSQL\CPI Inflation by State & Division.csv' CSV HEADER;
```

Figure 3.14 Uploading CSV file into PostgreSQL

These commands use the COPY statement to import cleaned datasets from CSV files into their respective database tables. The CSV HEADER option specifies that the first row of each CSV file contains column headers, ensuring accurate mapping of data to the table schema. This process facilitates the integration of multiple cleaned datasets into the database for further analysis and processing.

iii) Creating new table

```
CREATE TABLE district_data AS
SELECT

    a.state,
    a.district,
    a.year,
    a.income_mean,
    a.income_median,
    b.lf,
    b.lf_employed,
    b.lf_unemployed,
    b.lf_outside,
    b.p_rate,
    b.u_rate,
    b.ep_ratio,
    d.poverty_absolute,
    d.poverty_relative,
    e.stage AS enrollment_stage,
    e.sex AS enrollment_sex,
    e.students
FROM hh_income_district a
FULL OUTER JOIN lfs b
    ON a.state = b.state AND a.district = b.district AND a.year = b.year
FULL OUTER JOIN poverty_district d
    ON a.state = d.state AND a.district = d.district AND a.year = d.year
FULL OUTER JOIN enrollment_school_district e
    ON a.state = e.state AND a.district = e.district AND a.year = e.year;
```

Figure 3.15 Create new table for district_data

```sql
CREATE TABLE state_data AS
SELECT
    a.state,
    a.year,
    a.stage AS completion_stage,
    a.sex AS completion_sex,
    a.completion,
    b.division,
    b.inflation_yoy,
    b.inflation_mom
FROM completion_school_state a
FULL OUTER JOIN cpi_inflation b
    ON a.state = b.state AND a.year = b.year;
```

Figure 3.16 Create new table for state_data

```sql
CREATE TABLE final_data AS
SELECT

    a.*,
    b.completion_stage,
    b.completion_sex,
    b.completion,
    b.division,
    b.inflation_yoy,
    b.inflation_mom
FROM district_data a
FULL OUTER JOIN state_data b
    ON a.state = b.state AND a.year = b.year;
```

Figure 3.17 Create new table for combined data

The SQL queries demonstrate the process of integrating multiple datasets into a unified table for analysis. District-level data, including household income, labor force statistics, poverty measures and school enrollment, are combined into the `district_data` table using `FULL OUTER JOIN` to ensure no data is excluded, even if certain records are missing in one of the tables. Similarly, state-level data, such as school completion rates and inflation metrics, are merged into the `state_data` table. Finally, the `district_data` and `state_data` tables are joined to create the `final_data` table, which serves as a comprehensive dataset containing socioeconomic indicators at both district and state levels. The use of `FULL OUTER JOIN` ensures the inclusion of all data points, even if specific values are missing in one or more datasets, thereby maintaining completeness and consistency for further analysis and modeling.

iv) Saving The Combined Data

```sql
COPY final_data TO 'C:\Program Files\PostgreSQL\final_data.csv' CSV HEADER;
```

Figure 3.18 Saving combined data

This command uses the COPY statement to export the contents of the final_data table into a CSV file located at the specified path (C:\Program Files\PostgreSQL\final_data.csv). The CSV HEADER option ensures that the first row of the CSV file contains the column names from the table, making the

data more readable and easier to interpret when opened in external applications such as Excel or other data analysis tools.

3.2.3 After Combining Dataset

```
final_data = pd.read_csv('final_data.csv')
final_data
```

	state	district	year	income_mean	income_median	lf	lf_employed	lf_unemployed	lf_outside	p_rate	...	poverty_relative	enrollment_stage	enrollment_sex	students	completion_stage	completion_sex	completion	division	inflation_yoy	inflation_mom
0	Johor	Kota Tinggi	2019.0	6982.0	5475.0	100.7	98.0	2.8	43.8	69.7	...	20.8	post_secondary	male	621.0	primary	both	97.7	Personal Care, Social Protection & Miscellaneo...	-2.7	0.2
1	Johor	Kota Tinggi	2019.0	6982.0	5475.0	100.7	98.0	2.8	43.8	69.7	...	20.8	post_secondary	male	621.0	primary	female	98.2	Personal Care, Social Protection & Miscellaneo...	-2.7	0.2
2	Johor	Kota Tinggi	2019.0	6982.0	5475.0	100.7	98.0	2.8	43.8	69.7	...	20.8	post_secondary	male	621.0	primary	male	97.2	Personal Care, Social Protection & Miscellaneo...	-2.7	0.2
3	Johor	Kota Tinggi	2019.0	6982.0	5475.0	100.7	98.0	2.8	43.8	69.7	...	20.8	post_secondary	male	621.0	primary	both	97.7	Clothing & Footwear	-3.4	0.1
4	Johor	Kota Tinggi	2019.0	6982.0	5475.0	100.7	98.0	2.8	43.8	69.7	...	20.8	post_secondary	male	621.0	primary	female	98.2	Clothing & Footwear	-3.4	0.1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
220916	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	post_secondary	both	600.0	NaN	NaN	NaN	NaN	NaN	NaN
220917	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	post_secondary	female	442.0	NaN	NaN	NaN	NaN	NaN	NaN
220918	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	post_secondary	female	396.0	NaN	NaN	NaN	NaN	NaN	NaN
220919	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	post_secondary	male	255.0	NaN	NaN	NaN	NaN	NaN	NaN
220920	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	post_secondary	male	204.0	NaN	NaN	NaN	NaN	NaN	NaN

220921 rows × 23 columns

Figure 3.19 Review of dataset

Here is the overview of the dataset after integrating six datasets. Displaying these few rows provides insight into its structure, data type and potential values within the dataset. We also can observe several rows with missing values which will require further attention for the following analysis.

```
[ ]  # Convert 'year' column to integer, handling potential errors
     final_data['year'] = pd.to_numeric(final_data['year'], errors='coerce').astype('Int64')
```

Figure 3.20 Format feature

This function attempts to convert the values in the year column to integer format and code errors=’coerce’ replace any non-numeric value with NaN for more control later in handling the data inconsistencies.

```
duplicates = final_data.duplicated()
print(f"Number of duplicate rows: {duplicates.sum()}")
```

```
Number of duplicate rows: 368
```

```
[ ]  final_data = final_data.drop_duplicates()
```

```
[ ]  duplicates = final_data.duplicated()
     print(f"Number of duplicate rows: {duplicates.sum()}")
```

```
Number of duplicate rows: 0
```

Figure 3.21 Handle duplicate values

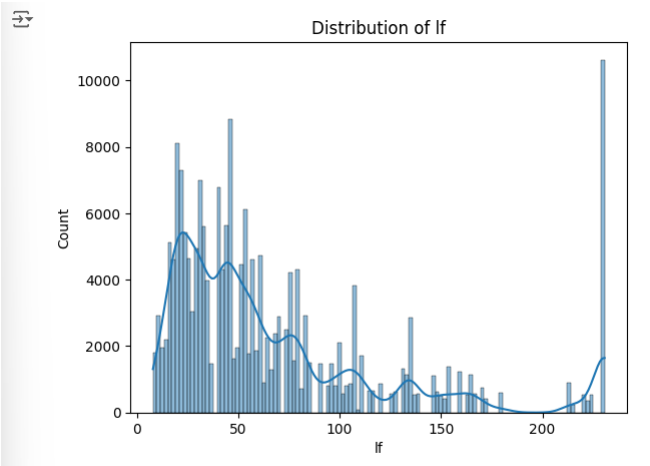
The duplicated().sum() function identifies the number of duplicate rows by counting those with identical values across all columns. In this case, 368 duplicate rows were found. Subsequently, these duplicates were removed and a recheck confirmed the absence of any remaining duplicates.

```
print(final_data.isnull().sum())
```

```
state                5563
district             5563
year                 5563
income_mean          5563
income_median        5563
lf                  23651
lf_employed         23651
lf_unemployed       23651
lf_outside          23651
p_rate              23651
u_rate              23651
ep_ratio            23651
poverty_absolute     5563
poverty_relative     5563
enrollment_stage     5299
enrollment_sex       5299
students             5299
completion_stage     2632
completion_sex       2632
completion           2632
division             2666
inflation_yoy        2666
inflation_mom        2666
dtype: int64
```

Figure 3.22 Checking missing values

The figure above shows the number of missing values for each column in the final_data DataFrame. The highest missing values in columns lf, lf_employed, lf_unemployed, lf_outside, p_rate, u_rate, ep_ratio with 23651 rows. Columns state, district, year, income_mean, income median, proverty_absolute, proverty_relative, enrollment_stage, enrollment_sex, students consist of more than 5000 missing value and completion_stage, completion_sex, division, inflation_yoy, inflation_mom with around 2000 missing value.




```
[ ]  final_data['lf'] = final_data['lf'].fillna(final_data['lf'].median())
```

Figure 3.23 Checking distribution

To handle the missing values, we analyze the distribution of attributes using a histogram chart. If the distribution is skewed, we impute the missing value with a median. It is because it is preferred

over the mean because skewed distributions can be influenced by outliers, leading to inaccurate imputation meanwhile if the distribution is normal distribution the missing value will imputed with means. For example, the distribution of the labor force (lf) is skewed to the right. Then we impute to improve the missing value with a median.

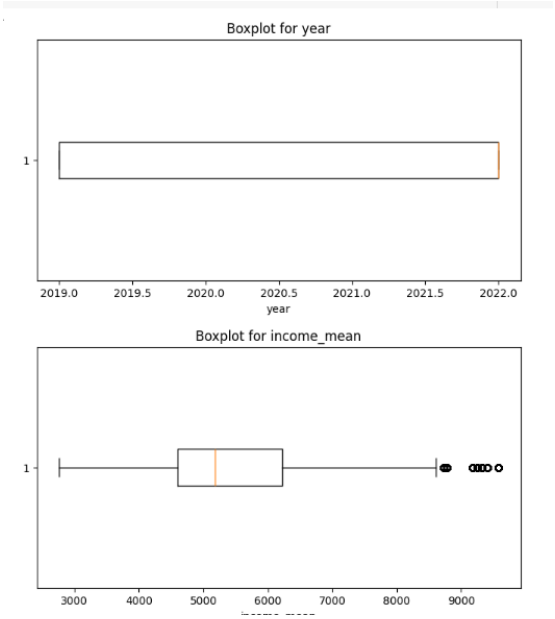


Figure 3.24 Boxplot for outliers

```
[44] def remove_outliers_iqr(final_data, columns):
         for column in columns:
             Q1 = final_data[column].quantile(0.25)
             Q3 = final_data[column].quantile(0.75)
             IQR = Q3 - Q1
             lower_bound = Q1 - 1.5 * IQR
             upper_bound = Q3 + 1.5 * IQR
             final_data = final_data[(final_data[column] >= lower_bound) & (final_data[column] <= upper_bound)]
         return final_data

     # Apply the function to remove outliers
     numerical_columns = final_data.select_dtypes(include=['float64', 'int64']).columns
     final_data= remove_outliers_iqr(final_data, numerical_columns)
```

Figure 3.25 IQR method

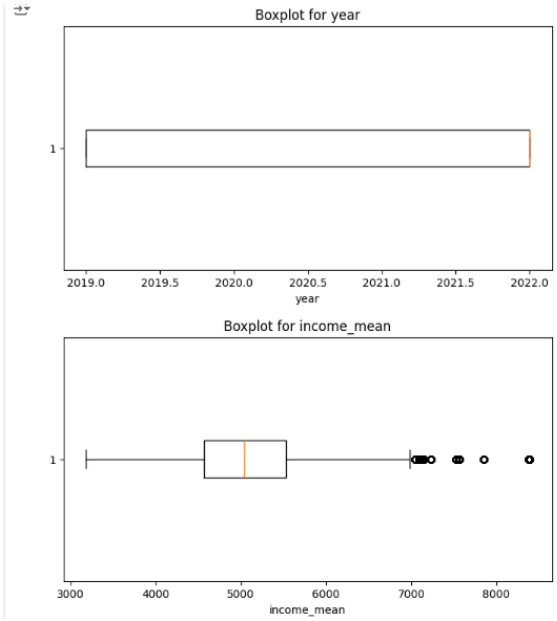


Figure 3.26 Boxplot after remove outliers

To identify outliers, we employed box plot analysis. The code calculates the 25th percentile and 75th percentile for each numerical attribute determining the interquartile range (IQR). Any data point exceeding 1.5 times the IQR above Q3 or below Q1 is outliers. The number of outliers detected in each column is then recorded before removing them due to their potential negative impact on the modeling.

```
[ ]  # Identify categorical columns
     categorical_columns = final_data.select_dtypes(include=['object']).columns
     print("Categorical Columns:", categorical_columns)

     Categorical Columns: Index(['state', 'district', 'enrollment_stage', 'enrollment_sex',
            'completion_stage', 'completion_sex', 'division'],
           dtype='object')

[ ]  label_encoder = LabelEncoder()

     # Apply Label Encoding to 'state', 'district', 'enrollment_stage', and 'completion_stage'
     final_data['state'] = label_encoder.fit_transform(final_data['state'])
     final_data['district'] = label_encoder.fit_transform(final_data['district'])
     final_data['enrollment_stage'] = label_encoder.fit_transform(final_data['enrollment_stage'])
     final_data['completion_stage'] = label_encoder.fit_transform(final_data['completion_stage'])
     final_data['enrollment_sex'] = label_encoder.fit_transform(final_data['enrollment_sex'])
     final_data['completion_sex'] = label_encoder.fit_transform(final_data['completion_sex'])

[ ]  final_data = pd.get_dummies(final_data,columns=[ 'division'],drop_first=True )
```

Figure 3.27 Label encode dataset

We identify all the columns in the dataset containing categorical data by selecting those with the object data type. We found 7 attributes which include state, district, enrollment_stage, completion_stage, enrollment_sex, completion_sex and division. The label encoding was applied to all categorical columns except division. This technique converts the categorical labels into numerical representations. State and district were labeled encoded since they have a large number of unique values. One hot encoding of these columns would have resulted in a significant increase in the number of attributes. Encollment_stage and completion _stage are ordinal variables that are suitable for label encoding. enrollment_sex, completion_sex, only have two classes which are female and male, and are also suitable for label

encoding. Meanwhile, for division, we performed one hot encoding. This creates a new binary column for each unique category. Each binary column indicates whether the observation belongs to that specific category (1) or not (0).

```
bin_edges = [0, 10, 20, float('inf')]
labels = ['Low', 'Moderate', 'High']

final_data['poverty_category'] = pd.cut(final_data['poverty_relative'], bins=bin_edges, labels=labels, include_lowest=True)
final_data
```

old gs, t & nce	division_Housing, Utilities, Gas, & Other Fuels	division_Information & Communication	division_Insurance & Financial Service	division_Personal Care, Social Protection & Miscellaneous Goods and Services	division_Recreation, Sport & Culture	division_Restaurant & Accommodation Services	division_overall	poverty_category
rue	False	False	False	False	False	False	False	High
rue	False	False	False	False	False	False	False	High
alse	False	True	False	False	False	False	False	High
alse	True	False	False	False	False	False	False	High
alse	True	False	False	False	False	False	False	High
...	...	...	...	...	...	...	...	...
alse	False	False	True	False	False	False	False	Low
alse	False	False	True	False	False	False	False	Low

Figure 3.28 Binning the target variable

Here we create a new variable called poverty_category column which categorizes the poverty_relative column into 3 groups which are low, moderate, and high. The categorization is based on low from 0 to 10, moderate from 10 to 20 and high is more than 20.

```
[ ]  final_data['poverty_category'] = label_encoder.fit_transform(final_data['poverty_category'])
```

Figure 3.29 Label encode target variable

This code converts the categorical labels in poverty_category column which are low moderate and high into numerical representations 0,1,2. It is required to perform machine learning since it only reads the numerical input.

```
[ ]  #Assign target variable
     X = df.drop(columns=['poverty_category'])
     y = df['poverty_category']
```

```
[ ]  # Split into numerical and categorical features
     numerical_features = X.select_dtypes(include=['int64', 'float64']).columns
     categorical_features = X.select_dtypes(include=['bool', 'category']).columns
```

Figure 3.30 Assigning variables

The code indicates separates the target variable and independent varriable. The poverty_category column is assigned as the target variable which is the variable we aim to predict. The remaining columns in the dataset are assigned to x as the independent variables. We also split the attributes in x into two groups which are numerical feature and categorical feature.

	Feature	F-Score	P-Value
13	poverty_relative	202182.276234	0.000000e+00
12	poverty_absolute	10759.039204	0.000000e+00
4	income_median	7996.299191	0.000000e+00
3	income_mean	6610.911520	0.000000e+00
8	lf_outside	5358.011827	0.000000e+00
5	lf	3435.007610	0.000000e+00
6	lf_employed	3421.368778	0.000000e+00
9	p_rate	2661.925129	0.000000e+00
11	ep_ratio	2426.478349	0.000000e+00
7	lf_unemployed	2316.061426	0.000000e+00
0	state	1957.192747	0.000000e+00
2	year	1619.451286	0.000000e+00
16	students	1071.612227	0.000000e+00
19	completion	704.469860	1.070993e-304
10	u_rate	483.837632	6.414306e-210
1	district	323.152898	1.187036e-140
17	completion_stage	114.636141	1.848807e-50
14	enrollment_stage	47.240171	3.110405e-21

Figure 3.31 Anova output

The code performs feature selection for numerical input using the Anova statistical test. Attributes with high f-score and low p-values indicate a strong association with the target variable. Therefore, we select the attributes that have a p-value less than 0.05.

	Feature	Chi-Score	P-Value
10	division_Restaurant & Accommodation Services	334.103521	2.820605e-73
11	division_overall	274.362212	2.648516e-60
2	division_Food & Beverages	263.289599	6.720322e-58
4	division_Household Furnishings, Equipment & Ma...	246.377694	3.160556e-54
3	division_Health	124.328515	1.005555e-27
9	division_Recreation, Sport & Culture	68.167681	1.576073e-15
8	division_Personal Care, Social Protection & Mi...	52.660804	3.671569e-12
1	division_Education	40.182367	1.881525e-09
5	division_Housing, Utilities, Gas, & Other Fuels	38.239325	4.970913e-09
6	division_Information & Communication	37.388032	7.608378e-09
0	division_Clothing & Footwear	24.071457	5.928566e-06
7	division_Insurance & Financial Service	1.083219	5.818111e-01

Figure 3.32 Chi-squared output

The code performs feature selection for categorical input using the chi-squared test. Attributes with high chi-square indicate a strong association between attributes with the target variable. A low p-value, usually a threshold like 0.05 suggests that the attributes are statistically significant.

```
# Select top features based on a threshold
selected_numerical = anova_results[anova_results['P-Value'] < 0.05]['Feature'].tolist()
selected_categorical = chi_results[chi_results['P-Value'] < 0.05]['Feature'].tolist()
```

```
# Combine selected features
selected_features = selected_numerical + selected_categorical
print("\nSelected Features:")
print(selected_features)

# Create final dataset
X_selected = df[selected_features]
```

Figure 3.33 Combine selected features

We filter the attributes based on their p-value from ANOVA and Chi-square test that must be less than 0.05 to be significant attributes. Then we combine these selected numerical and categorical features and create a new dataset called df with only the selected features. This helps to reduce the dimensionality of the data by selecting only the most relevant features.

```
# Convert all `bool` columns to integers
bool_columns = df.select_dtypes(include='bool').columns
df[bool_columns] = df[bool_columns].astype(int)

# Check the updated data types
print(df.dtypes)
```

```
state                                                                  int64
district                                                               int64
year                                                                   Int64
income_mean                                                          float64
income_median                                                        float64
lf                                                                   float64
lf_employed                                                          float64
lf_unemployed                                                        float64
lf_outside                                                           float64
p_rate                                                               float64
u_rate                                                               float64
ep_ratio                                                             float64
poverty_absolute                                                     float64
poverty_relative                                                     float64
enrollment_stage                                                       int64
enrollment_sex                                                         int64
students                                                             float64
completion_stage                                                       int64
completion_sex                                                         int64
completion                                                           float64
inflation_yoy                                                        float64
inflation_mom                                                        float64
division_Clothing & Footwear                                           int64
division_Education                                                     int64
division_Food & Beverages                                              int64
division_Health                                                        int64
division_Household Furnishings, Equipment & Maintenance                int64
division_Housing, Utilities, Gas, & Other Fuels                        int64
division_Information & Communication                                   int64
division_Insurance & Financial Service                                 int64
division_Personal Care, Social Protection & Miscellaneous Goods and Services    int64
division_Recreation, Sport & Culture                                   int64
division_Restaurant & Accommodation Services                           int64
division_overall                                                       int64
poverty_category                                                       int64
dtype: object
```

Figure 3.34 Checking data type for each variable

All Boolean columns in the df data frame are converted into integer values with 1 indicating True and 0 for False. The conversion can be necessary for certain machine learning algorithms or data analysis techniques that require numerical input. The data type of the df dataset was rechecked by code dtypes that now have an integer and numerical datatype only.

```
[ ]  #Split dataset to train and test using Holdout method
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30)
```

Figure 3.35 Splitting the dataset

The dataset was split since it is crucial in machine learning to evaluate the performance of the model. In this case, 30% of the data will be allocated to the test set and the remaining 70% will be used for training.

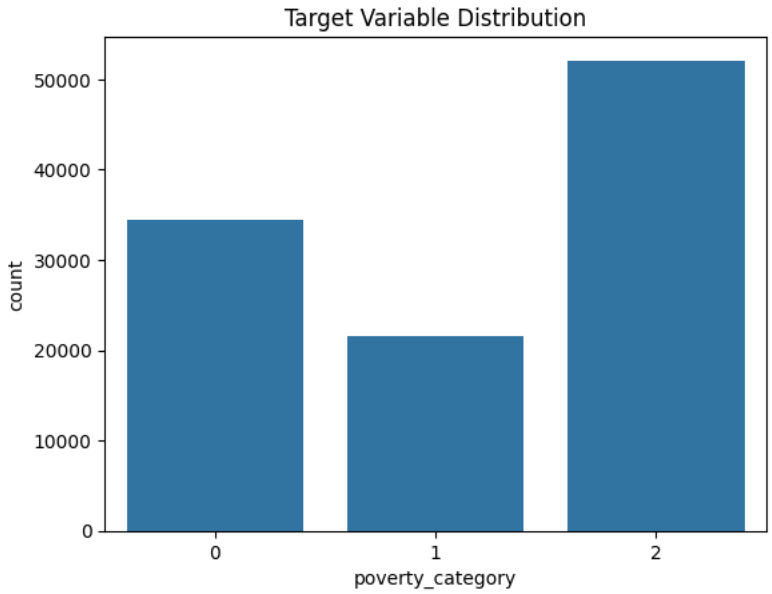


Figure 3.36 Target variable distribution

The resulting plot is a bar chart that visualizes the distribution of the poverty category variable. Each bar is the class of the variable, and the height of the bar represents the number of counts belonging to that class. For example, we can see that there are more instances in category 2 compared to Category 0 or 1. This information can be useful for understanding the class imbalance in the data.

```
Class distribution after SMOTE:
poverty_category
0    36344
2    36344
1    36344
Name: count, dtype: int64
```

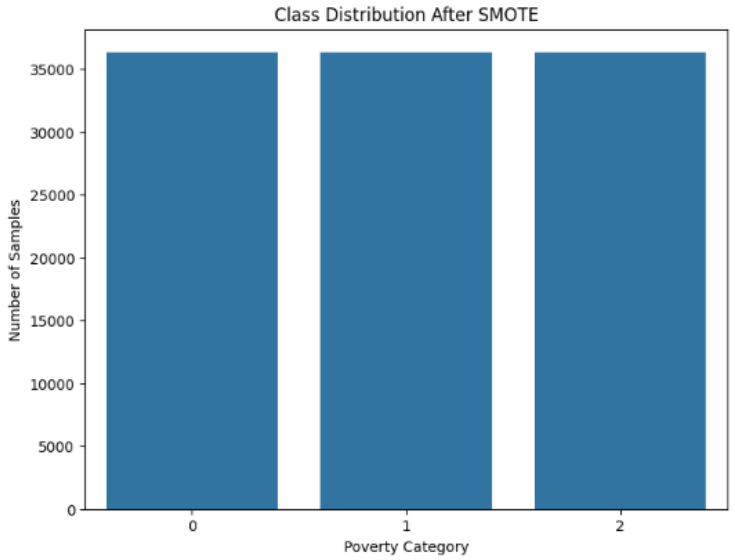


Figure 3.37 Class distribution after SMOTE

The plot shows the class distribution of the target variable after applying SMOTE. Smote is a popular oversampling technique that generates synthetic samples for the minority class to balance the class distribution. Therefore, now each category consists of 36344 instances.

```
[ ]  scaler = StandardScaler()
     X_train_scaled=scaler.fit_transform(X_train)
     X_test_scaled=scaler. fit_transform(X_test)
```

Figure 3.38 Feature scaling

We also perform feature scaling on both training and testing data using StandardScaler. it standardizes features by removing the mean and scaling to unit variance. This ensures that all features have a similar scale, which can improve the performance.

```
[ ]  pca = PCA(n_components=2, random_state=24)
     X_train_scaled = pca.fit_transform(X_train_scaled)
     X_test_scaled = pca.transform(X_test_scaled)
```

Figure 3.39 PCA implementation

We implemented PCA to reduce the dimensionality of the scaled training and test data to two principal components. This means that the original data, which has many features, will be projected onto a 2-dimensional subspace. PCA will reduce the number of features, which can improve model performance by simplifying the model and reducing the risk of overfitting and also shorten time process the tuning hyperparameter Gridsearch.

3.3 Exploratory Data Analysis

3.3.1 Poverty Rate vs Unemployment Rate

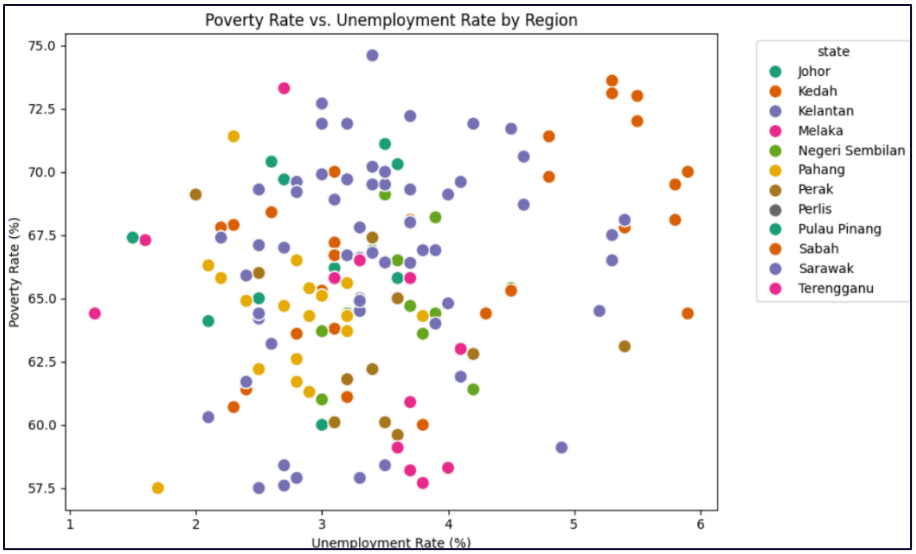


Figure 3.40 Scatter plot poverty rate

Interpretation:

Based on above scatter plot, it shows the relationship between the poverty rate (%) and the unemployment rate (%) across various state in Malaysia. Each state is represented by the color and the data point are scattered which shows no strong linear correlation between both variables. Poverty rates range between 57.5 % to 75% while the unemployment rates between 1% to 6%. In this visualization, states like Kelantan and Sabah show higher poverty rates with moderate unemployment rates. It may be due to factors such as rural area and depends on agriculture, so it gives limit access to quality education and skills training then lead to lower economic. Meanwhile, states like Johor and Pulau Pinang show lower poverty rates because of their growing economies, it lead to better access to education, government investment.

3.3.2 Average Income by State

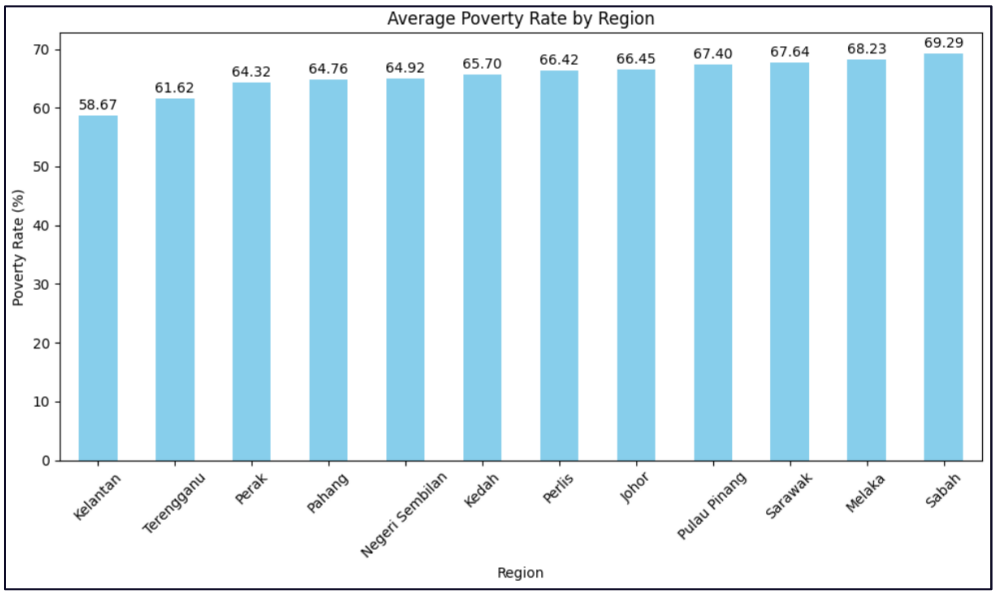


Figure 3.41 Bar chart average income by state

Interpretation:

This bar chart shows the average poverty rate (%) by region in Malaysia, with Kelantan having the lowest average poverty rate (58.67%) and Sabah the highest (69.29%). The variation in poverty rates is affected by regional factors. States like Kelantan and Terengganu are more rural, but they may have a stronger community systems or lower living cost that can lead to reduce the poverty levels. On the other hand, Sabah's high poverty rate can be attributed to limited infrastructure, lower access to education and healthcare and economic dependence on agriculture and natural resources. Some developing states like Johor and Pulau Pinang shows a moderate poverty rates due from the better employment opportunities and type of economic activities there, but they are not the lowest because living costs can still be a burden to lower income households.

3.3.3 Income Means vs Students

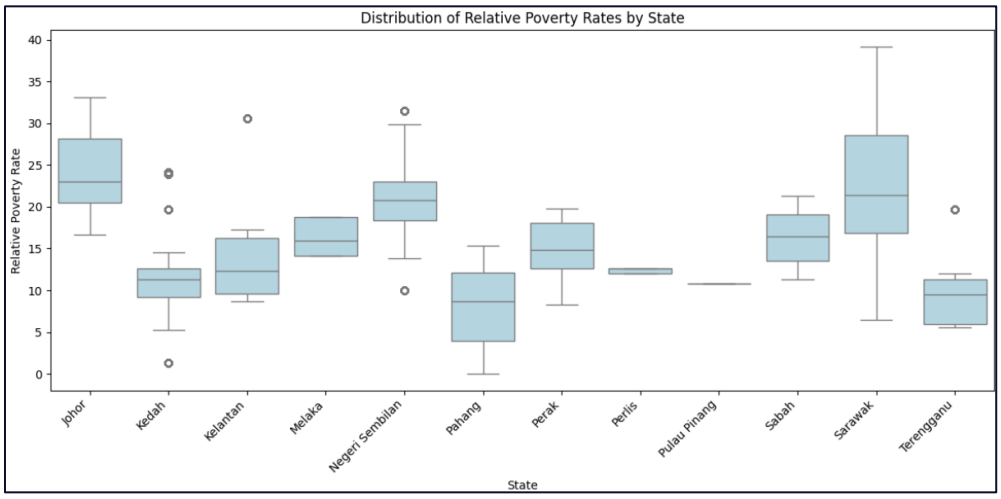


Figure 3.42 Boxplot of income means and students

Interpretation:

Above box plot shows the distribution of relative poverty across states in Malaysia. As we can see Sarawak has the highest variability in poverty rates, its indicate that significant level within these both states, it may be due to rural and remote areas so its experience severe poverty than other states. Meanwhile states like Johor have high median poverty rates around 25% but narrow distribution, suggest that the poverty level is consistent among their populations. In contrast, Pulau Pinang and Perlis have lower median poverty rates with small variability and Pahang shows the lowest median poverty rate. As for the outliers, we can see there are several of them at Kedah, Kelantan, Negeri Sembilan and Terengganu, it needs to be treated by identifying the areas that may need further measure to address the poverty in those states.

3.3.4 Employment vs Relative Poverty

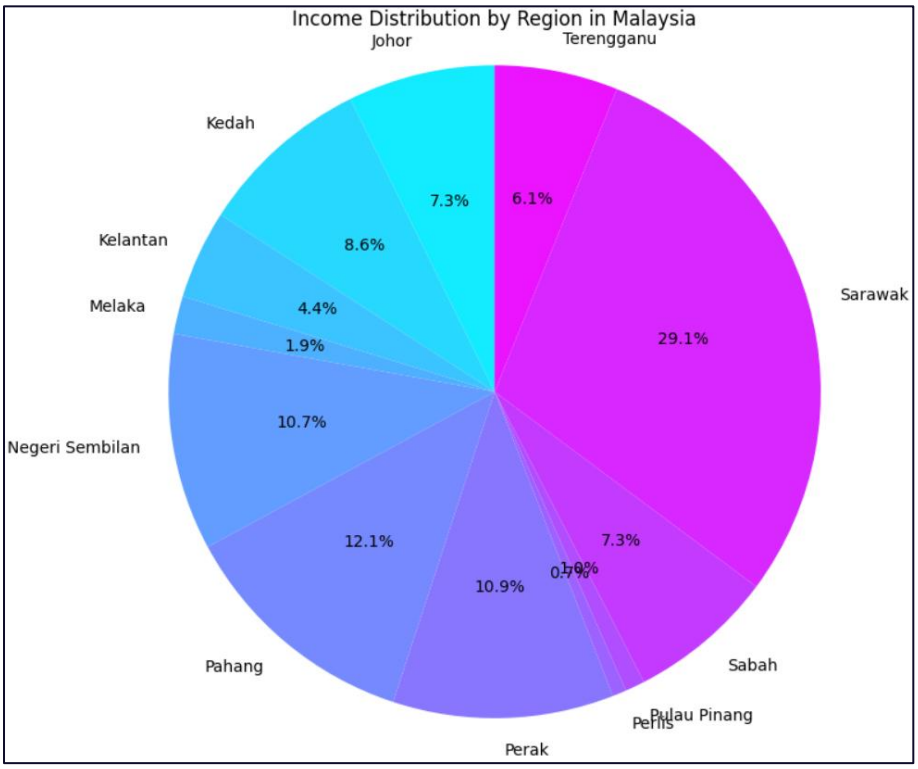


Figure 3.43 Pie chart of employment with poverty by region

Interpretation:

Above visualization shows pie chart for income distribution by region in Malaysia. It shows that the largest distribution is from Sarawak 29.1 %, followed by Pahang 12.1 % and 10.7% from Negeri Sembilan. The smallest distribution comes from Perlis and Pulau Pinang which respectively 0.7% and 1.0%. This shows that most income was from Sarawak as we know that one of the economic activities from that state is oil and gas, as the federal government considers Sarawak's probable and proven reserves of petroleum to be 60.87% of entire Malaysia(). While Perlis is has the smallest percentage distribution because it is one of the smallest state in Malaysia, with fewer population it can lead to smaller income distribution compared to larger states.

3.3.5 Proportion of Enrollment Stages

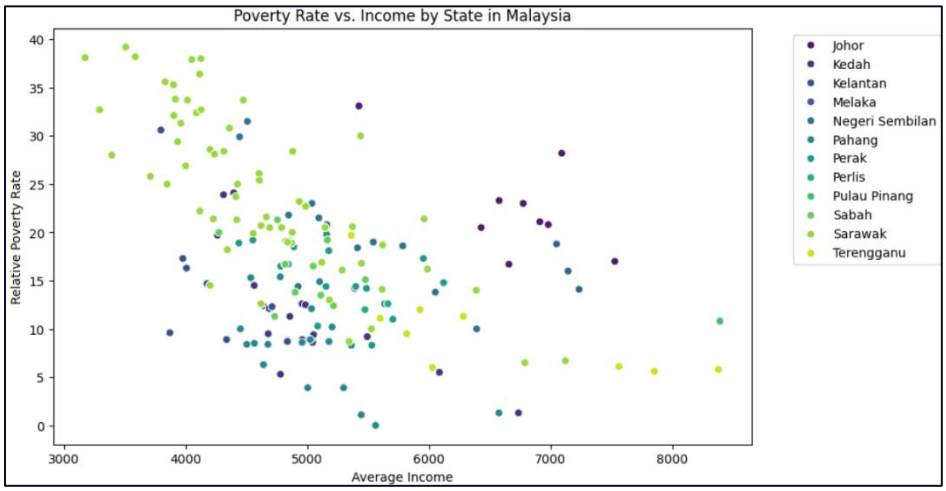


Figure 3.44 Scatter plot of relative poverty rates and average income by region

Interpretation:

This scatter plot shows the relationship between relative poverty rates and average income across Malaysian states. Generally, there is a downward trend, indicating that states with higher average incomes tend to have lower poverty rates. For example, states like Johor and Pulau Pinang, which exhibit higher average incomes (above 6000), have relatively low poverty rates. Conversely, states like Kelantan and Sabah, with lower average incomes (around 3000-5000), show higher poverty rates, some exceeding 30%. The dispersion of data points also suggests that while income level is a key factor influencing poverty, other socio-economic factors, such as access to education, infrastructure and employment opportunities, contribute to variations within and between states. This highlights the need for targeted involvements to reduce poverty in lower-income regions.

4.0 METHODOLOGY

4.1 Tools Used

a) Pandas
Pandas can be used to do data exploration, cleaning and manipulation as an example handling missing values, remove the outlier and duplicates.

b) Matplotlib
Matplotlib can be used to create various plots like box plot, scatter plots, bar charts and pie charts to visually represent data.

c) Seaborn
This library can be used to create histogram plots.

d) Sklearn
Scikit-learn can be used for data preprocessing, model selection, and evaluation. Through Scikit-Learn, it can encode categorical variables, handle missing values in the dataset, split the data into training and testing sets, and evaluate model performance using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE) and R-square.

e) Imblearn
The library is used for unbalanced classifications.

f) Xgboost
Optimized distributed gradient boosting library designed to be highly efficient, flexible and portable.

g) TensorFlow
TensorFlow can be used to define, train, and evaluate neural networks, which are Artificial Neural Networks (ANNs).

h) Math
Math is a core Python library for basic mathematical functions. In our case study, the math package's square root function was used to calculate the RMSE.

i) NumPy
NumPy is a powerful numerical computing library focusing on array and linear algebra. It is used for array manipulation and operation functions for tasks like one hot encoding, calculating AUC, and other numerical computations.

4.2 Modeling Pipeline

Figure 4.1 shows the architecture of our model. There are five models that we tried to fit to the algorithm which are decision tree, naïve bayes, KNN, XGBoost and ANN.

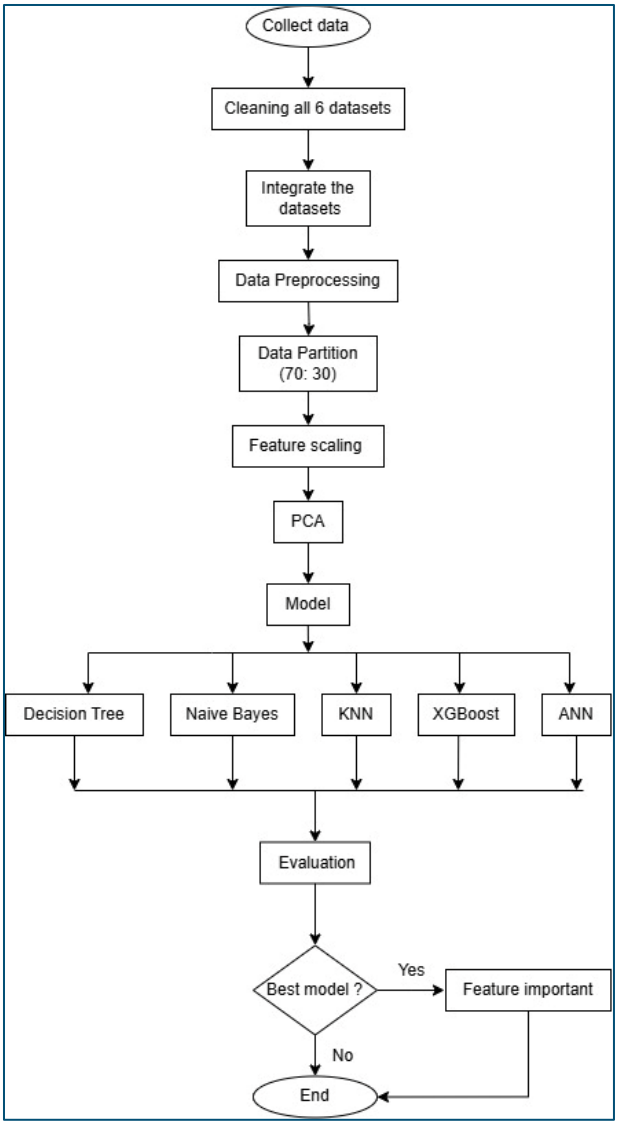


Figure 4. 1 Flowchart of the pipeline

1. Collect Data.
   This is the initial step where we gather data from various sources such as DOSM and Data.gov.

2. Cleaning all 6 datasets.
   Since real world datasets are often messy, we involve a data cleaning and preparing step for each dataset. For instance, we handle missing value, handle inconsistencies and remove irrelevant data such as we are only interested in data from 2019 to 2022.

3. Integrate the data
   As our data comes from multiple source, we need to combine all six dataset into one dataset. This involve tools like SQL. The process of combining the datasets must ensure each data point is aligned at common levels such as state, district and year.

4. Data Preprocessing
   We revisit the preprocessing process because some features might be missing from certain dataset led to missing value. We also perform other preprocessing task like cleaning, transformation into certain format that suitable for modeling and selecting relevant features.

5. Data partition
   We split the data into two data sets. The training dataset must be larger than testing where in our case we used ratio 70:30. It means that 70 % of the data will be used to train and other 30 percent used to evaluate how well it performs.

6. Feature scaling.
   This process is crucial in modeling where ensures all features in the dataset are a uniform scale. We uses a standard scaler because our dataset might contain outliers that could skew the model's learning process.

7. PCA.
   Pca or Principal component analysis might be necessary if our dataset are still large after preprocessing. PCA helps simplify data by identifying the most important features and reducing the number of dimension making model easier to learn

8. Model
   In this process we implement several machine learning model which are decision tree, naïve bayes, KNN, XGBoost, and ANN.

9. Evaluation
   Once the model are trained, we used the testing data to see how well it perform. Evaluation metrics include accuracy, classification report, confusion metric, ROC curve and AUC- ROC score.

10. Best model?
    Here, we make a decision based on the evaluation results. We compare the performance of all the model and select one that perform the best. The best model are used for next process to achieve our objective.

11. Feature Importance
    We perform a further analysis on the best model which to identify features that were most influential in making the predictions. This helps us understand which factor have the most significant impact on the model's outcomes.

12. End
    This marks the successful completion of our project.

4.3 Machine Learning Algorithms

4.3.1 Decision Tree

Decision trees stand out as highly effective machine learning models for both classification and regression tasks. Their strength lies in their interpretability, allowing users to easily understand the decision-making process, and their versatility in handling both numerical and categorical data (Sivananda & Kumar, 2024). Despite their advantages, overfitting can pose a significant challenge, particularly with complex datasets (García Leiva et al., 2019). To maximize the performance and generalization of decision trees, meticulous hyperparameter tuning is crucial. Research indicates that employing optimization techniques can greatly boost the accuracy of decision tree models, even if the average improvement is modest (Mantovani et al., 2016). Innovatively, a new approach has been introduced that construct decision trees designed to avoid overfitting, achieving high accuracy without the need for extensive hyperparameter optimization (García Leiva et al., 2019). In real-world applications, decision trees have proven to be remarkably accurate, exemplified by a striking 99.89% accuracy achieved in air quality classification tasks following hyperparameter tuning (Sabella & Pristyanto, 2024). Ultimately, decision trees are not just a powerful tool in machine learning; they are an essential asset, delivering accurate and interpretable models that can be applied across a wide range of domains.

```
dt = DecisionTreeClassifier(random_state=42)

param_grid = {
       'criterion': ['gini', 'entropy'],
       'max_depth': [None, 5, 10, 15, 20],
       'min_samples_split': [2, 5, 10],
       'min_samples_leaf': [1, 2, 4]
             }

grid_search = GridSearchCV(estimator=dt,
                           param_grid=param_grid,
                           cv=5,  # Number of cross-validation folds
                           scoring='accuracy',  # Evaluation metric
                           verbose=1)
```

```
grid_search.fit(X_train_scaled, y_train)
best_params = grid_search.best_params_
best_dt_model = grid_search.best_estimator_
print(f"Best parameters: {best_params}")
```

```
[ ] dt_model = DecisionTreeClassifier(criterion = 'entropy', max_depth= 15, min_samples_leaf = 1, min_samples_split = 2, random_state=42)
    dt_model.fit(X_train_scaled, y_train)

    # Predict on test data
    y_pred_dt = dt_model.predict((X_test_scaled))

    # Evaluate
    print("Decision Tree Results:")
    print("Accuracy:", accuracy_score(y_test, y_pred_dt))
    print("Classification Report:\n", classification_report(y_test, y_pred_dt))
```

Figure 4.2 Decision Tree Model

In this study, the implementation of the Decision Tree model started with the strategic definition of the base model using DecisionTreeClassifier(random_state=42). To maximize its potential, a grid search approach was utilized via GridSearchCV to meticulously optimize hyperparameters. This grid encompassed key variables such as criterion (entropy and Gini) for evaluating impurities, max_depth to combat overfitting, min_samples_split, and min_samples_leaf which determine the thresholds for data splitting. Employing five-fold cross-validation ensured a thorough evaluation across diverse data subsets. This rigorous grid search process is crucial in guaranteeing that our model operates at its best. The chosen hyperparameters criterion=entropy, max_depth=15, min_samples_split=2, and min_samples_leaf=1 was precisely fine-tuned to align with the dataset's unique characteristics. Once these optimal parameters were discerned, the model was retrained on the scaled training data, and its effectiveness was meticulously assessed using accuracy metrics and other classification measures. This approach not only reinforces the model's reliability but also enhances its predictive power, ultimately delivering superior results.

4.3.2 Naïve Bayes

Naïve Bayes algorithm is a probabilistic model that is based on the Bayes Theorem and conditioned under the independence of features, it has been used mostly in doing classification (Chauhan, 2022). This algorithm was selected due to its simplicity and effectiveness in handling probabilistic classification tasks. It performs when there is conditional independence among the features in the dataset (Chauhan, 2022). This algorithm is a simple probabilistic classifier and has a few parameters to be used in building machine learning models that can give prediction faster than other classification model (Choudhary, 2024).

```
# Initialize and train Naive Bayes
nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train)

# Predict on test data
y_pred_nb = nb_model.predict(X_test_scaled)


# Evaluate
print("Naive Bayes Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Classification Report:\n", classification_report(y_test, y_pred_nb))
```

Figure 4.3 Naïve Bayes Model

Figure above shows code for Naïve Bayes algorithm used in this project. In here we just used the default Naïve Bayes code and use Gaussian algorithm to train a classification model. We first initialize the nb_model and train the model by using scaled training data. The model then predicts the labels for scaled test data and evaluates its performance by using evaluation metrics like accuracy and classification report. The results will show overall accuracy, precision, recall and F1- score for each class of poverty category.

4.3.3 K-Nearest Neighbor (KNN)

The KNN algorithm is one of the supervised learning classifiers, which use the nearness of data to make predictions or classification about grouping of individual data point. This algorithm is one of the famous and simple classification and regression classifiers that we used today in machine learning (IBM, 2021). It can handle both numerical and categorical data and also less sensitive to outlier compared to other machine learning algorithms. The KNN algorithm will work by find out the K-nearest neighbours to a given data point based on a distance metric, such as Euclidean distance (GeeksforGeeks, 2024).

```
#GridSearchCV
knn = KNeighborsClassifier()
k_range = list(range(1, 40)) #list smpai 40 je  ikut je sng
param_grid = dict(n_neighbors=k_range)

# defining parameter range
grid = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy',
                    return_train_score=False,verbose=1)
# fitting the model for grid search
grid_search=grid.fit(X_train_scaled, y_train)
print(grid_search.best_params_)
```

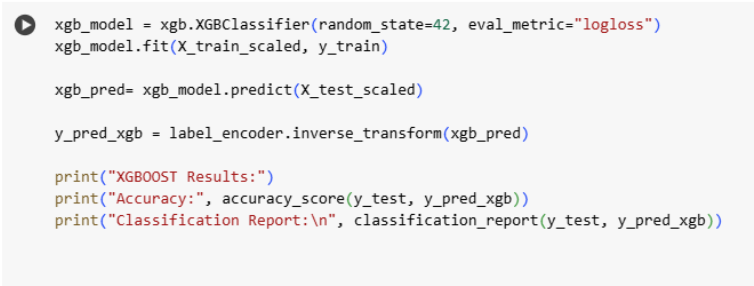
```
knn_model = KNeighborsClassifier(n_neighbors = 15, metric='euclidean')
knn_model.fit(X_train_scaled, y_train)

# Predict on test data
y_pred_knn = knn_model.predict((X_test_scaled))
```

Figure 4.4 KNN Model

Based on figure above it shows code for KNN algorithm. This code implements GridSearchCV to optimize the hyperparameters for this classifier. The range of possible values for n_neighbours parameter are from 1-40 and use GridSearchCV to search for the best parameter combination through a cross validation (cv) = 5 and optimizing for accuracy. Th best parameter then will be printed. After identifying the optimal n_neighbors value = 15 and using metrics, Euclidean distance, the KNN model is trained on the scaled training data and used to predict the label for scaled test data.

4.3.4 XGBoost

XGBoost or extreme Gradient Boosting is a be highly efficient, flexible, and portable machine learning framework (XGBoost Documentation, 2022). It has gained significant popularity for its ability to build a strong predictive model by aggregating the predictions of several weak learners (ML | XGBoost (eXtreme Gradient Boosting), 2023). Fundamentally, XGBoost constructs each weak learner sequentially, learning from and correcting the mistakes made by previous learners (ML | XGBoost (eXtreme Gradient Boosting), 2023). This iterative approach ensures continual improvement in the model's performance.One of the standout benefits of XGBoost is its incorporation of regularization techniques. These techniques help combat overfitting, enabling the model to generalize effectively to unseen data (Crossman-Smith, 2024). This balance between performance and generalization makes XGBoost a reliable choice for a wide range of predictive modeling tasks.



```
xgb_model = xgb.XGBClassifier(random_state=42, eval_metric="logloss")
xgb_model.fit(X_train_scaled, y_train)

xgb_pred= xgb_model.predict(X_test_scaled)

y_pred_xgb = label_encoder.inverse_transform(xgb_pred)

print("XGBOOST Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print("Classification Report:\n", classification_report(y_test, y_pred_xgb))
```

Figure 4. 5 XGBoost Model

Here the implementation of XGBoost algorithm for our case study. The parameter used in the XGbClaasifier consist of random_state=42 that set the seed used within the XGBoost algorithm. With this setseed if we run the code many times the result will be the same. The next second parameter eval_metric=logloss is the evaluation metric to be used during the training process. It calculates the average negative log-likelihood of the predicted probabilities. This metric helps the model to minimize the logloss and potential to improve its overall performance.

4.3.5 Artificial Neural Network (ANN)

ANN is a powerful deep learning algorithm that mimics the human brain's structure and processes in a computer way. The works of ANN are almost similar to human brain neural networks, although ANN algorithm only accepts numerical and structured data from user. The neurons within interconnected units will combine to identify the patterns, get knowledge from data and then will generate the predictions. There are three layers in this ANN network architecture; input, hidden (more than 1) and output layer. The activation function is needed because it contributes to conversion of input to usable final output, example of activation function is ReLU and Sigmoid (Singh, 2021).

```
# Encode target labels
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)

num_classes = len(le.classes_)  # Number of unique classes
```

```
# Define the model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=34, activation='relu', input_shape=(X_train_scaled.shape[1],)),  # Input layer
    tf.keras.layers.Dense(units=17, activation='relu'),  # Hidden layer
    tf.keras.layers.Dense(units=num_classes, activation='softmax')  # Output layer for multi-class
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',  # For multi-class classification with integer labels
    metrics=['accuracy']
)

# Fit the model with EarlyStopping
early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=10, restore_best_weights=True
)

history = model.fit(
    X_train_scaled, y_train_encoded,
    batch_size=32,
    epochs=100,
    validation_split=0.2,
    callbacks=[early_stopping],
    verbose=1
)
```

Figure 4.6 ANN Model

Based on the above code, it shows the ANN algorithm. Firstly, the LabelEncoder is used to convert categorical variables in y_train and y_test to encode integers, then stored in num_classes. In this model is a feedforward ANN. Secondly, it then will process the data in a single direction, then move sequentially from input layer through hidden layers to output layers. The architecture of this model used fully connected layers using Dense layers in Keras, where input layer is 34 units and hidden layer with 17 units by using ReLU activation. An output layer with num_classes units and softmax activation for multiclass classification. Then, compiling the model using Adam optimizer, sparse categorical cross entropy loss since it suitable for integer labels and accuracy as evaluation metrics. It then trains and fit the model using Early Stopping to prevent overfitting, monitoring validation loss with a patience 10. Lastly, the training use a batch size of 3, runs up to 100 epochs and splits the data of validation for 20%. Early stopping will ensure the best weight is restored after stopping.

5.0 RESULTS AND CONCLUSION

5.1 Interpretation of Results

In this section, we will interpret the results from the predictive model we have developed to ensure choosing the right and valid model to validate which features contribute most to poverty in order to provide actionable solutions in overcoming the poverty.

5.1.1 Model Performance Metrics

i) **Decision Tree classification report and confusion matrix**

```
Decision Tree Results:
Accuracy: 0.7491595472349875
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.82      0.82     10237
           1       0.58      0.78      0.67      6444
           2       0.81      0.69      0.75     15742

    accuracy                           0.75     32423
   macro avg       0.73      0.76      0.74     32423
weighted avg       0.77      0.75      0.75     32423
```

Figure 5.1 Decision Tree Classification Report

The Decision Tree model demonstrated an accuracy of 74.92%, successfully predicting nearly three-quarters of the test instances. The classification report reveals essential insights into the model's performance across different classes. Notably, precision was highest for class 0 (High) which is at 0.81, indicating the model's reliability in predicting this category accurately. Additionally, recall which captures the proportion of actual positive cases identified was robust for both classes 0 and 2 (Moderate), both achieving a score of 0.81. This indicates that the model excels in identifying most instances in these categories. However, class 1 (Low) exhibited a recall of only 0.58, suggesting that the model faces challenges in recognizing all instances in this group. The F1-score, which harmonizes precision and recall, further reflects overall model performance, with the macro average at 0.73 suggesting consistent capabilities across classes. These results not only highlight the model’s strengths in predicting dominant categories but also indicate critical opportunities for enhancement, particularly in addressing minority classes that require more precise identification.

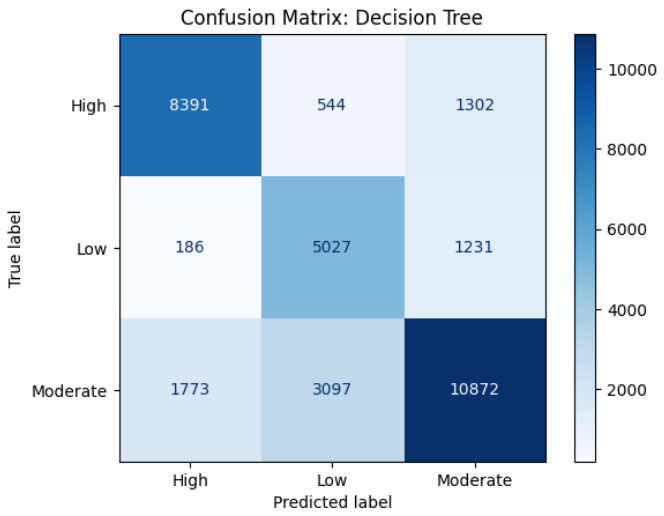


Figure 5.2 Decision Tree Confusion Matrix

The confusion matrix serves as a compelling visual tool that showcases the Decision Tree model's performance by contrasting actual and predicted labels across three key classes which are High, Low and Moderate. The diagonal elements, indicating 8,391 correctly classified High, 5,027 Low and 10872 Moderate, highlight where the model excels. These results are a clear testament to the model's strength. However, the off-diagonal values reveal areas for improvement through misclassifications. For instance, 1,302 cases of the High class were mistakenly identified as Moderate, and 3,097 Moderate instances were misclassified as Low. These inaccuracies underscore the challenges the model faces in differentiating between closely related classes. While the model performs admirably for the majority class, Moderate, evident from the higher correct predictions, it struggles more with distinguishing between High and Moderate, as well as Low and Moderate.

ii) **Naïve Bayes classification report and confusion matrix**

```
Naive Bayes Results:
Accuracy: 0.4521481664250686
Classification Report:
              precision    recall  f1-score   support

           0       0.57      0.71      0.63     10237
           1       0.32      0.71      0.44      6444
           2       0.52      0.18      0.26     15742

    accuracy                           0.45     32423
   macro avg       0.47      0.53      0.45     32423
weighted avg       0.50      0.45      0.42     32423
```

Figure 5.3 Naïve Bayes classification report

The Naive Bayes model recorded an accuracy rate of 45.25%, which indicates that it successfully predicted fewer than half of the instances. This modest accuracy is clearly reflected in the classification report, which summarizes the precision, recall, and F1-score for each category. Remarkably, for class 0 (High), the model achieved the best F1-score of 0.63, supported by a precision of 0.57 and a recall of

0.71. This suggests a considerable ability to correctly identify instances in this category. However, the model encountered substantial difficulties with class 1 (Low) and class 2 (Moderate), which returned F1-scores of 0.44 and 0.28, respectively. This gap highlights the challenges faced in accurately classifying the less prevalent classes. Furthermore, with a macro average F1-score of 0.45 and a weighted average of 0.42, the overall results demonstrate that the performance of the Naive Bayes model is not meeting expectations.

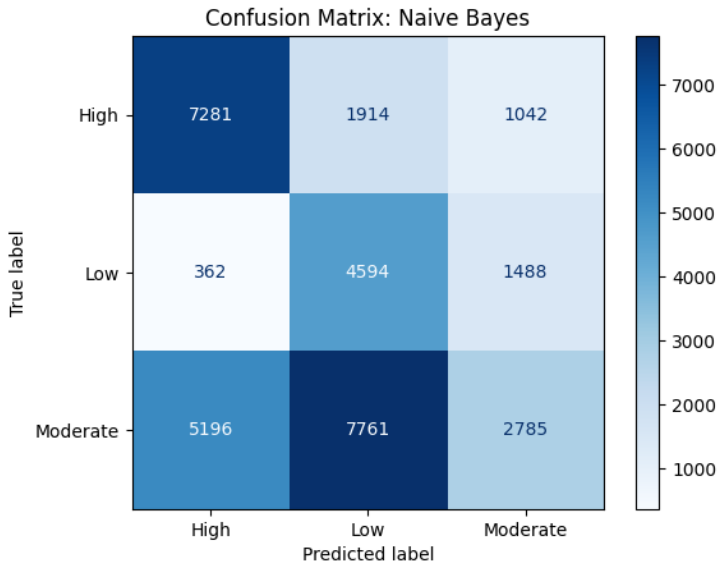


Figure 5.4 Naïve Bayer Confusion Matrix

The confusion matrix for the Naive Bayes model provides essential insights into its classification performance for the three key labels such as High, Low, and Moderate. The diagonal elements illustrate successes, with 7,281 instances correctly classified as High, 4,594 as Low and 2,785 as Moderate. However, the off-diagonal elements reveal concerning misclassifications that need to be addressed. For instance, 1,914 instances of High were incorrectly labeled as Low, while a notable 7,761 instances of Moderate were misclassified the same way. This pronounced bias towards the Low class suggests issues such as imbalanced datasets or overlapping features that demand attention. Additionally, 5,196 Moderate instances were wrongly classified as High, complicating the model's ability to distinguish between categories.

iii) **KNN classification report and confusion matrix**

```
KNN WITH GRIDSEARCH Results:
Accuracy: 0.7678808253400364
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.84      0.83     10237
           1       0.61      0.80      0.69      6444
           2       0.83      0.71      0.76     15742

    accuracy                           0.77     32423
   macro avg       0.75      0.78      0.76     32423
weighted avg       0.78      0.77      0.77     32423
```

Figure 5.5 KNN Classification Report

The accuracy of the K-Nearest Neighbours (KNN) model, which was optimised with GridSearch, was 76.78%, or approximately three-quarters of the predictions were accurate. The classification report breaks down performance by class. The model's greatest performance was for class 0, with an F1-score of 0.83, precision of 0.82 and recall of 0.84. The model had more trouble correctly detecting occurrences in class 1, as seen by its lower precision of 0.61 and F1-score of 0.69. Class 2's performance was balanced, with an F1-score of 0.76, precision of 0.83 and recall of 0.71. The weighted average F1-score of 0.77 takes into consideration the distribution of classes, whilst the macro average F1-score of 0.78 represents the model's overall performance across all classes, weighted equally.

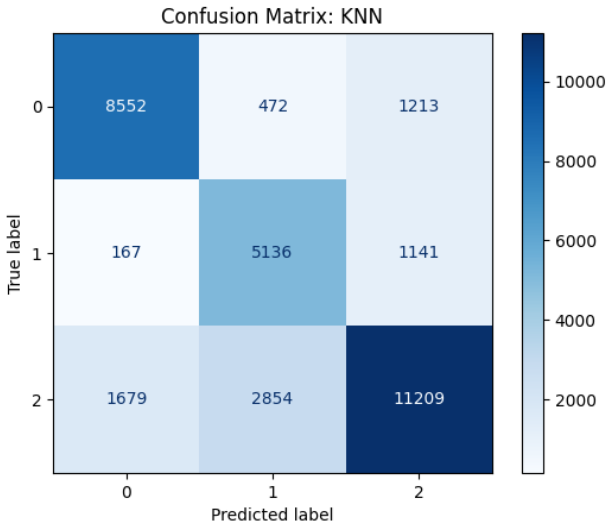


Figure 5.6 KNN Confusion Matrix

The confusion matrix shows how successfully each class was classified by the KNN model of the 10,237 real cases for class 0, 8,552 were correctly classified, 472 were incorrectly classified as class 1, and 1,213 as class 2. The 6,444 incidents in class 1, 5,136 were correctly identified. However, 167 were incorrectly classified as class 0 and 1,141 as class 2. Finally, the model successfully identified 11,209 out of 15,742 cases for class 2, while misclassifying 1,679 and 2,854 as class 0 and class 1,

respectively. According to these findings, the model performs well in class 0 but more poorly in class 1 and class 2, suggesting that these classes may overlap.

iv) **XGBoost classification report and confusion matrix**

```
XGBOOST Results:
Accuracy: 0.7721062208925763
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.84      0.83     10237
           1       0.62      0.79      0.69      6444
           2       0.83      0.72      0.77     15742

    accuracy                           0.77     32423
   macro avg       0.76      0.78      0.76     32423
weighted avg       0.78      0.77      0.77     32423
```

Figure 5.7 XGBoost Classification Report

With an accuracy of 77.21%, the XGBoost model outperformed the KNN model by a small margin, meaning that around 77 out of 100 predictions were accurate. Each class's specific performance metrics are provided in the categorization report. With a precision of 0.82 and a recall of 0.84, the model performed well for class 0, yielding an F1-score of 0.83. Class 1 achieved an F1-score of 0.69 by outperforming KNN in recall of 0.79 while maintaining comparable precision of 0.62. Class 2 achieved an F1-score of 0.76 with precision of 0.83 and recall of 0.72. A well-balanced performance across all classes is indicated by the weighted average F1-score of 0.77 and the macro average F1-score of 0.77. The marginally better accuracy and recall of XGBoost imply that it is more adept than KNN in identifying patterns in the data.

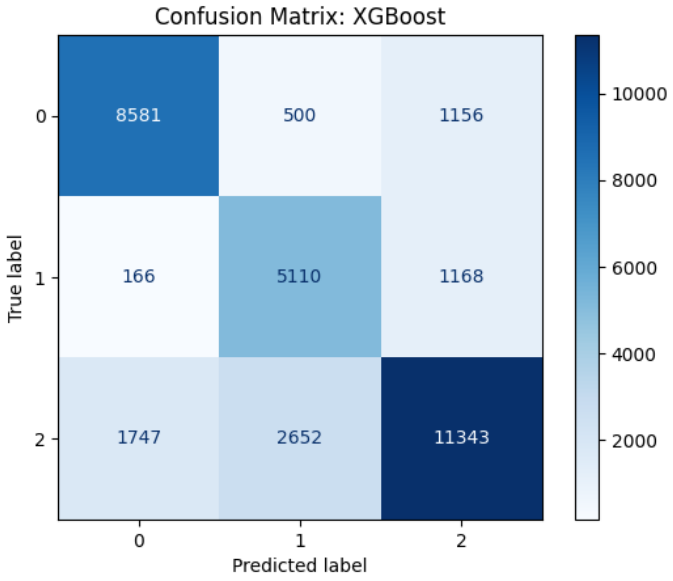


Figure 5.8 XGBoost Confusion Matrix

A split of XGBoost's true and anticipated classifications is given by the confusion matrix. 500 cases were incorrectly classified as class 1 and 1,156 as class 2, whereas 8,581 out of 10,237 instances were appropriately classified as class 0. 5,110 out of 6,444 class 1 cases were accurately identified by the model, but 166 were incorrectly classified as class 0 and 1,168 as class 2. Last but not least, of the 15,742 cases in class 2, 11,343 were correctly identified, while 2,652 were misclassified as class 1 and 1,747 as class 0. These findings show that while class 1 is still the most difficult because of data imbalances or overlaps, XGBoost performs well in class 0 and class 2, misclassifying less frequently than KNN.

v) **ANN classification report and confusion matrix**

```
ANN Results:
Accuracy: 0.7522746198686118
Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.86      0.82     10237
           1       0.63      0.67      0.65      6444
           2       0.79      0.72      0.75     15742

    accuracy                           0.75     32423
   macro avg       0.73      0.75      0.74     32423
weighted avg       0.75      0.75      0.75     32423
```

Figure 5.9 ANN Classification Report

With an accuracy of 75.23%, the Artificial Neural Network (ANN) model performed somewhat worse than XGBoost and KNN. The model's performance for every class is displayed in the classification report. With a precision of 0.78, recall of 0.86 and F1-score of 0.82 for class 0, the ANN demonstrated good performance, showing that the majority of class 0 occurrences were properly identified. With a precision of 0.63, recall of 0.67 and F1-score of 0.65, class 1 performed the worst, indicating difficulties in correctly identifying instances of this class. With a precision of 0.79, recall of 0.72, and F1-score of 0.75, class 2's outcomes were in balance. While the weighted average F1-score of 0.75 accounts for the distribution of classes, the overall average F1-score of 0.74 represents the performance across all classes equally.

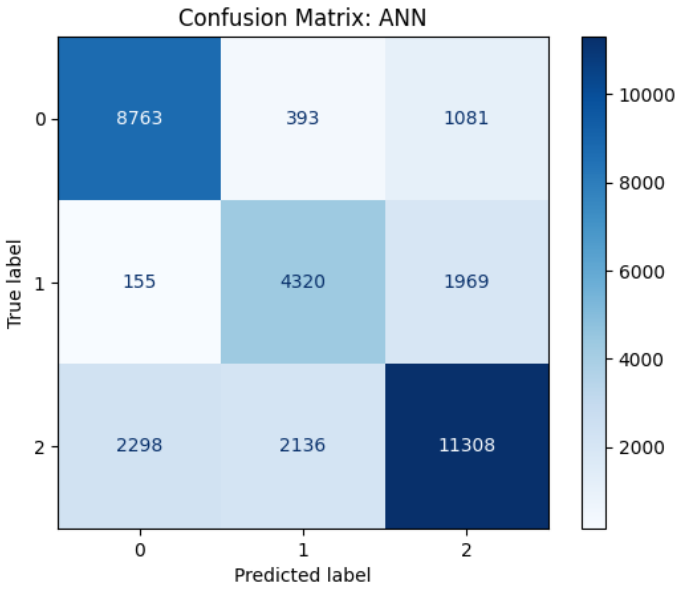


Figure 5.10 ANN Confusion Matrix

The confusion matrix for the artificial neural network (ANN) model offers critical insights into its classification outcomes. In class 0, the model identified 8,763 out of 10,237 true instances correctly, but it incorrectly labeled 393 as class 1 and 1,081 as class 2. For class 1, the model accurately classified 4,320 of the 6,444 instances yet misclassified 155 as class 0 and a significant 1,969 as class 2. Lastly, regarding class 2, the model correctly identified 11,308 out of 15,742 instances, though it misclassified 2,298 as class 0 and 2,136 as class 1. These findings highlight that the ANN model performs admirably for classes 0 and 2, but it falls short with class 1. This challenge likely stems from overlaps or imbalances within the dataset.

vi) **MSE, MAE and RMSE for each model**

Model	MSE	MAE	RMSE
NB	1.125035	0.740246	1.060677
ANN	0.560374	0.351942	0.748581
Decision Tree	0.535361	0.345681	0.731683
XGB	0.496499	0.317429	0.704627
KNN	0.483114	0.311261	0.695064

Figure 5.11 Error Metrics for each 5 Models

Based on the above table, it shows the comparison of three error metrics which is Mean Squared Errror (MSE), Mean Absolute Error (MAE) and Root Mean Square Error (MSE). The lower value indicates better model performance. From this, KNN has the lowest error values across all, making it the best-performing model in terms of accuracy and prediction error. While XGBoost is the second-best model with slightly higher errors, still showing strong performance. Lastly, Naive Bayes (NB) performs the worst, with significantly higher error values, suggesting it is not well-suited for this dataset. Overall, KNN have the most accurate predictions with the smallest errors, followed by XGBoost, while Naive Bayes performs worse than other models.

vii) ROC and AUC

a. Naïve Bayes

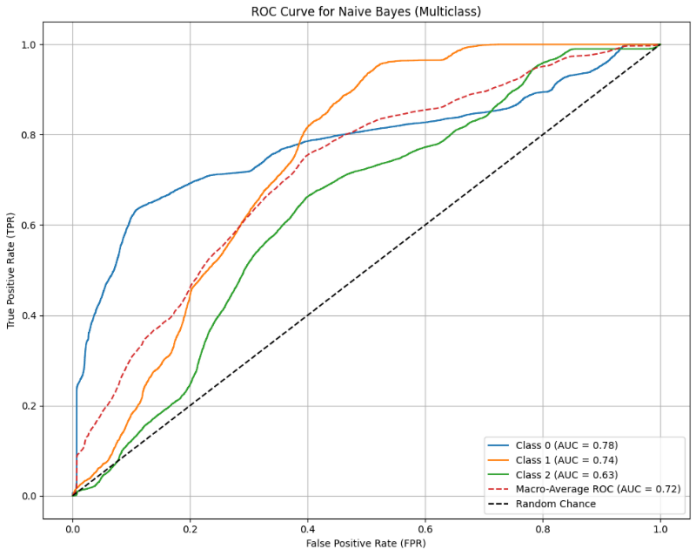


Figure 5.12 Naïve Bayes ROC & AUC

Figure 5.12 shows ROC and AUC for performance of a Naïve Bayes model. Class 0 has the best predictive performance with an AUC of 0.78, followed by Class 1 with an AUC of 0.75. Class 2 has the lowest performance, with an AUC of 0.64, indicating it is the most challenging class for the model to predict accurately. Based on the macro average AUC scores for all classes it shows overall perfomace of classes with 0.72, indicate that the Naïve Bayes model performs better than random chance for all classes.

b. KNN

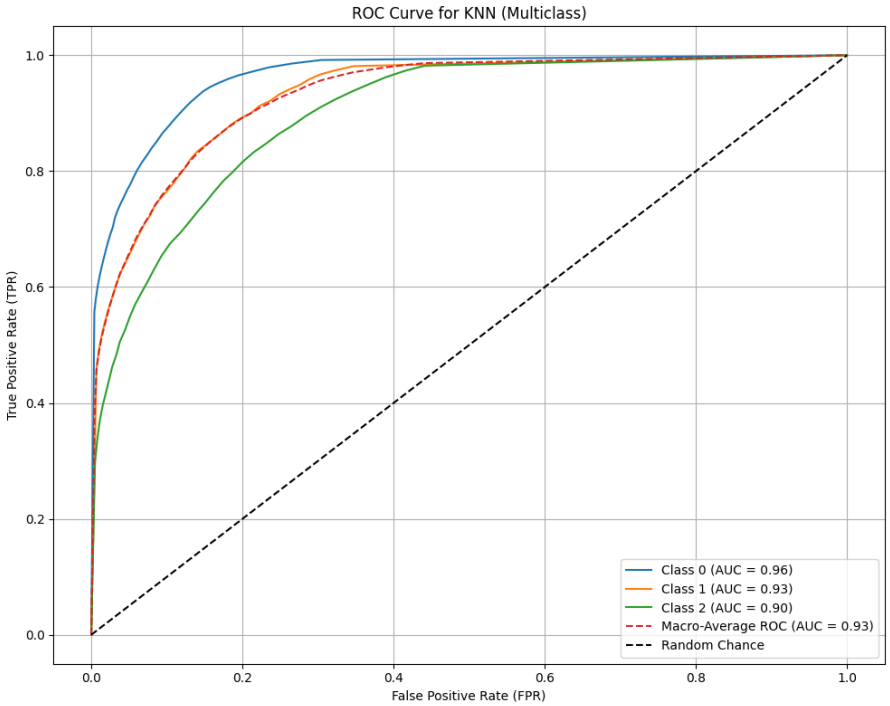


Figure 5.13 KNN ROC & AUC

Figure 5.13 shows KNN model performance across all the class classification. The model doing well by showing an AUC score of class 0.96 indicate excellent prediction of accuracy, followed by class 1 with 0.93 and lastly class 2 with an AUC of 0.90, shows that it is the worst class for model to predict it accurately. The macro average AUC score is 0.93, which reflects a strong overall performance for all the classes. While the ROC curves for all classes stays far from the random chance line shows that KNN model is highly effective at differentiate the different classes in this dataset.

c. XGBoost

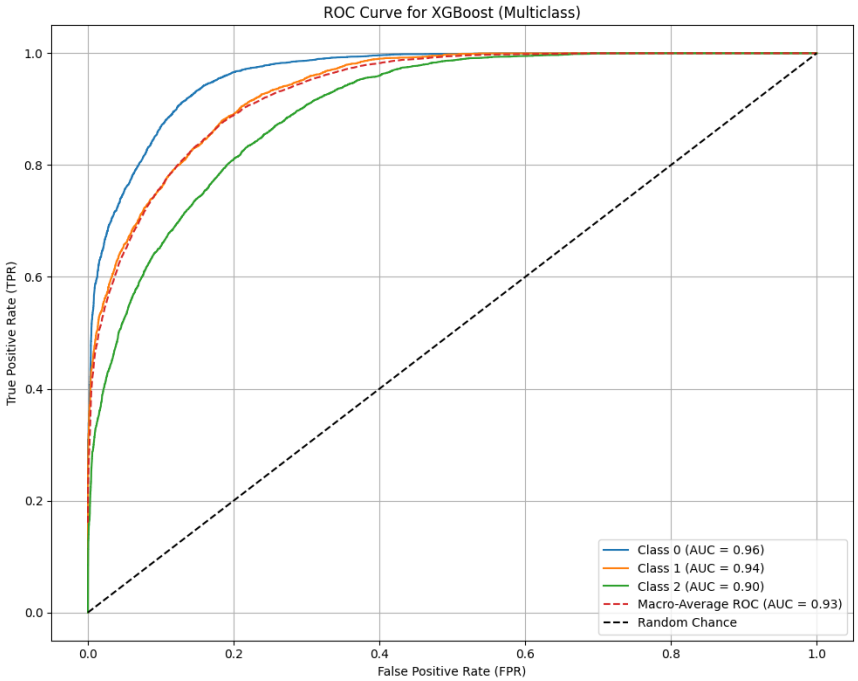


Figure 5.14 XGBoost ROC & AUC

Figure 5.14 shows ROC AUC curves for performance if XGBoost model for all classes classification. This model shows a good performance overall by having the curves of all classes stay above the random chance line, making XGBoost model a reliable choice for doing classification tasks for this project. As for the AUC score, class 0 achieve a most accurately prediction with 0.96 scores, followed by class 1 with an AUC of 0.94 and lowest class 2, 0.90. The macro average of AUC is 0.93 reflecting a good overall performance across 3 classes.

d. ANN

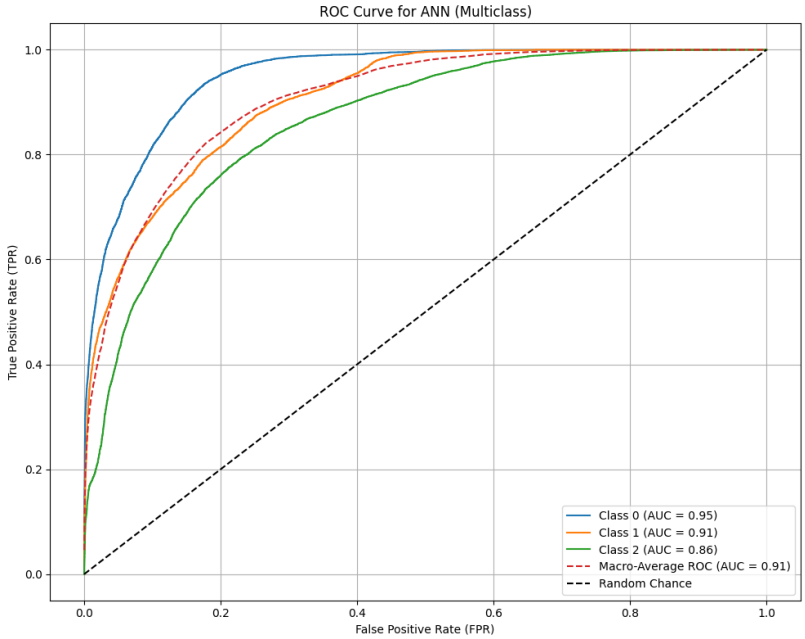


Figure 5.15 ANN ROC & AUC

Figure 5.15 shows the ROC AUC curve for ANN model for all classes. The model performs well overall, with class 0 is the highest 0.95 indicate a good performance in prediction. Followed by class 1 0.91 and class 2 with the lowest performance and an AUC of 0.86. The macro average AUC of 0.91 shows an excellent overall performance across all classes. The ROC curves for all classes are being far from the random chance line which indicate that, the ANN model is better and effective to differentiate between all classes and making it become a good model for this dataset.

e. Decision Tree

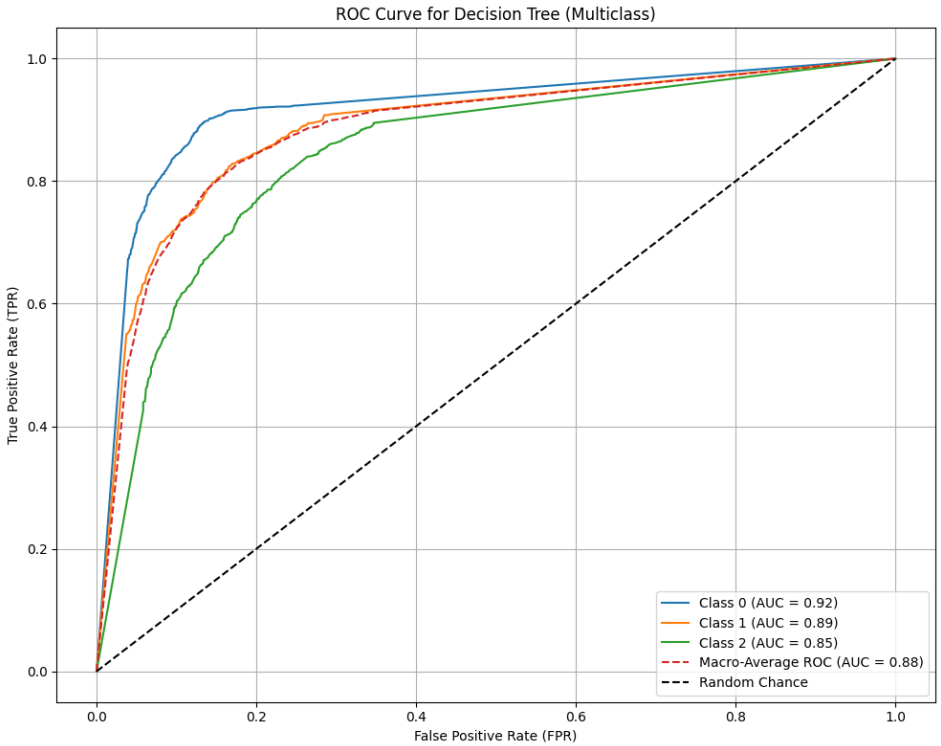


Figure 5.16 Decision Tree ROC & AUC

Figure 5.16 shows ROC AUC curve for decision tree classifiers performance between all classes. The AUC scores indicate a strong classification performance with class 0 the highest 0.92, followed by class 1 (0.89) and class 2 (0.85). The macro average AUC is 0.88 shows the overall performance across 3 classes. The curves of ROC are doing well and being above the random chance line (0.5), shows that the model is significantly better than random guess for all classes.

Model	Macro-Average AUC	AUC for Class 0	AUC for Class 1	AUC for Class 2
XGBoost	0.932858	0.962344	0.936109	0.900094
KNN	0.931568	0.962698	0.931874	0.900132
ANN	0.906288	0.949940	0.906541	0.862321
Decision Tree	0.883276	0.917135	0.887277	0.845405
Naive Bayes	0.718115	0.777946	0.742854	0.633466

Figure 5.17 ROC & AUC of each 5 Model

By looking at this comparison table of AUC score results, we can see that XGBoost model shows good performance compared to other models. It has highest macro average AUC, which is 0.93 that indicate a good performance in doing prediction all classes. As for individual classes, XGBoost also has a consistent high value of AUC scores, with class 0 at 0.96, class 1 at 0.94 and class 2 at 0.90. All of these scores can differentiate score between classes accurately.

The reason XGBoost performs better is because it uses an advanced boosting technique that improves its accuracy step by step. It is also very good at understanding complex patterns in the data and works well even when the data isn’t perfectly balanced. Compared to other models like Naive Bayes or KNN, XGBoost handles features and interactions more effectively, which helps it make better predictions. This makes it the most reliable choice based on the ROC AUC scores.

5.1.2 Findings Insights

To address the third objective for this project, the XGBoost predictive model was utilized to identify the key factors that influence poverty. By analyzing the feature importance scores derived from the model, we were able to determine the most significant variables contributing to the poverty levels. Figure 5.1.2 illustrates the top features, providing valuable insights into the main factors of poverty and laying the foundation for proposing actionable solutions.

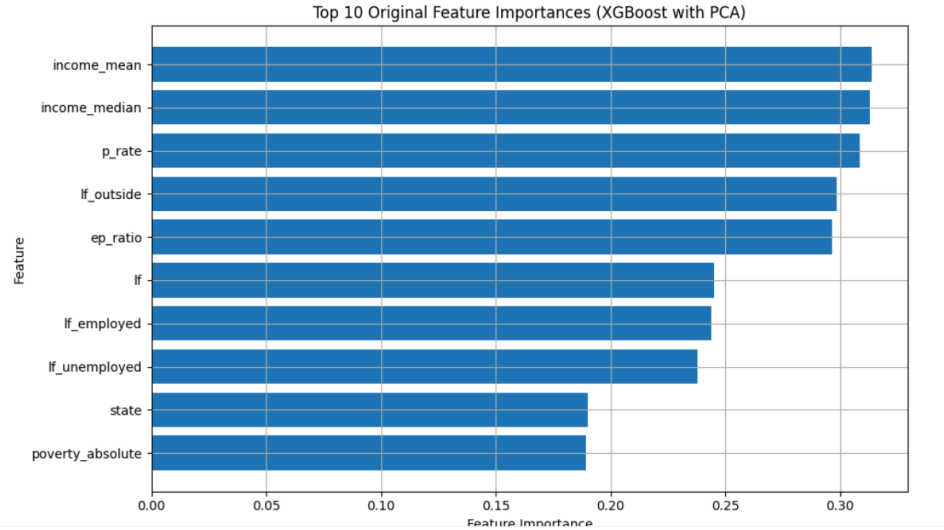


Figure 5.18 Feature Importance of XGBoost

	Feature	Importance
3	income_mean	0.313695
4	income_median	0.312872
9	p_rate	0.308434
8	lf_outside	0.298537
11	ep_ratio	0.296332
5	lf	0.245260
6	lf_employed	0.243994
7	lf_unemployed	0.237716
0	state	0.189936
12	poverty_absolute	0.189447
13	poverty_relative	0.163440
16	students	0.156400
10	u_rate	0.108171
32	division_Restaurant & Accommodation Services	0.059888
14	enrollment_stage	0.057902
19	completion	0.047007
17	completion_stage	0.035785
30	division_Personal Care, Social Protection & Mi...	0.032844
23	division_Education	0.031355
24	division_Food & Beverages	0.027290
22	division_Clothing & Footwear	0.025948
27	division_Housing, Utilities, Gas, & Other Fuels	0.017906
15	enrollment_sex	0.017631
28	division_Information & Communication	0.017052
29	division_Insurance & Financial Service	0.012419
31	division_Recreation, Sport & Culture	0.011860
25	division_Health	0.009117
21	inflation_mom	0.008548
1	district	0.007783
20	inflation_yoy	0.007029
26	division_Household Furnishings, Equipment & Ma...	0.006781
18	completion_sex	0.003195
33	division_overall	0.003110
2	year	0.002144

Figure 5.19 Feature Importance Score

The analysis of feature importance illustrated in Figure 5.1 and Figure 5.2 is derived from the XGBoost predictive model where it has provided significant insights into the factors contributing to poverty in Malaysia. The top features influencing poverty are primarily centered around income levels, labor force dynamics and regional disparities. Notably, the mean and median income emerged as the most critical features with the feature score of 0.3137 and 0.3129, reflecting the central role of financial stability in poverty alleviation. Higher income levels directly correlate with reduced poverty, this highlights the need for targeted interventions aimed at increasing household income.

The participation rate in the labor force represented by ‘p_rate’ and the number of people not currently working, which is the ‘lf_rate’ were found also to be the important factors where both have an importance score of 0.3084 and 0.2985 respectively. This means that encouraging more people to join the workforce and reducing unemployment are essential steps to reduce poverty. Additionally, the percentage of employment- population which is the ‘ep_ratio’, 0.29963 score who have jobs shows that creating more job opportunities and helping people find work are the key areas to focus on.

Regional disparities also play a significant role in poverty, as illustrated in figure 5.1.2 which is represented by the ‘state’ feature aligning with its score of 0.1899. This underscores the need of tailoring poverty alleviation programs to meet the specific needs of each region and understanding that the challenges faced by the rural areas differ from the urban areas is also important. Additionally, the indicator of absolute poverty, 0.1894 score where it measures the difficulty of poverty highlights the need to provide quick and targeted help to the poorest communities.

Lower ranking features such as education related variables and variables related to specific industries, suggest that while these factors influence poverty their impact is less obvious compared to income and employment related features. Nonetheless, these findings highlight potential areas for long-term investment such as improving education access and expanding opportunities in specific sectors.

5.1.3 Actionable Insights

The findings of the feature importance analysis from Figure 5.1 and 5.2 derived from the XGBoost predictive model, several actionable solutions have been proposed to address the key factors influencing poverty in Malaysia. These recommendations are data-driven and tailored to the target areas with the greatest potential impact, aligning with the fourth objective of this project. Below are the key insights and corresponding solutions:

1. **Immediate relief for absolute poverty**
   Insight: The feature poverty_absolute highlights the urgency of addressing poverty where individuals or families are unable to meet the basic needs for their daily life.

   Recommendations:
   1. Emergency food and nutrition programs by establishing food banks, mobile food distribution units and school meal programs in high poverty areas to address hunger and malnutrition.
   2. Direct Cash Assistance where we could collaborate with NGOs or any other organizations to provide targeted one-time or short-term cash transfers to families in a critical state of poverty. This will empower them to prioritize and meet their immediate needs such as food, housing, healthcare and utilities.

2. **Regional**
   Insight: The state feature highlights those regional disparities that play a significant role in the poverty levels. These suggest that different states or regions face unique challenges where some may experience higher poverty rates.

   Recommendations:
   1. Custom solutions for each region by addressing the specific needs of rural and urban areas by improving infrastructure in the rural regions and reducing the living costs in cites area.
   2. Regional employment hubs where it creates job centers in the underdeveloped regions in order to provide more job opportunities. Other than that, provide training to teach people the skills they need for local jobs and encourage business to set up in these areas by offering tax cuts or financial help.

3. **Income enhancement**
   Insight: Income- related factors such as income_mean and income_median are the strongest predictors of poverty. Higher income levels directly correlate with reduced poverty where they highlight the critical role of financial stability.

Recommendations:

1. Community-Based work opportunities where any organization can set up projects in high poverty areas such as maintaining the local parks or running community kitchens. Thus, the unemployed people can work and earn an income.
2. Develop a Work and Earn internship program for adults, our company can partner with government or private companies to create a short term paid internship program for adults with limited skills. This can help them earn money while gaining skills that could lead to a stable job.

4. **Labor force participation and employment**

Insight: Workforce related factors such as participation rate (p_rate), labor force outside employment (lf_outside), underscores the importance of getting more people into workforce and creating stable job opportunities.

Recommendations:

1. Create a "First Job Support Program" for youth by partnering with companies to create entry-level positions that are specifically designed for first time job seekers. This program can provide mentorship and on the job training to polish and enhance their skills.
2. Offer flexible work arrangements where it promotes remote work opportunities, especially for parents and other individuals with caregiving responsibilities. This will allow more individuals to join the workforce without having to compromise between family or personal needs.

5. **Education**

Insight: Education related factors such as students and completion rate play a supportive role in poverty reduction. While it is not the strongest prediction in the short term, education has a significant impact for a long term on improving job opportunities and income levels, thus leading to reducing poverty.

Recommendations:

1. Digital learning access by providing or a low-cost access to digital learning platforms, computers and internet services for students in rural or remote areas.
2. Educational Infrastructure improvements by building or upgrading the school areas in rural and underdeveloped regions to ensure that they also have access to modern facilities, qualified teachers and sufficient learning resources.

5.2 Conclusion & Recommendations

As for the conclusion, this project successfully achieved its objective which is to identify the poverty in Malaysia as example employment, household and education by identifying the regional disparities, building machine learning model for the predictions and proposing an actionable solution for this targeted poverty. By using several machine learning models, XGBoost was proved to be the excellent model to perform better, by achieving the highest macro average AUC scores (0.93). While KNN showed a lower error rate, XGBoost model was known as its ability to provide detailed insights through the feature importance analysis, which was critical to study goals.

Based on the key findings, it shows that income related factors such as average and median household income, were strongest determinants of poverty. These findings highlight the essential role of financial stability in reducing the poverty in Malaysia. Moreover, employment and workforce were identified as one of the critical areas that require further intervention to create a stable job opportunity among people in Malaysia. Other than that, this study also seeks in the regional disparities with rural areas that facing infrastructure challenges and urban areas with high living costs. Lastly, education access appears as a long-term factor in reducing the poverty in this category by improving job opportunities and income levels by the times. By integrating this data driven insights with actionable solutions, this study can contribute to a better understanding of poverty in Malaysia.

For future improvement and recommendations, expanding this project and analysis to include some additional factors such as healthcare access or environmental conditions can provide a better and more understanding of poverty in various area. Developing a user-friendly dashboard for real time data visualization would empower policymakers and stakeholders to make decisions efficiently based on the insight gains. Additionally, by doing targeted interventions such as launching emergency food programs, establish regional employments hub, and improve digital and physical educational infrastructure are important to addressing the immediate needs in solving this poverty challenges. By doing a collaborations with government sectors, NGOs and local communities can helps to ensure this data driven solution are effectively implement, fostering unbiased development and sustainable poverty across Malaysia.

REFERENCES

Crossman-Smith, J. (2024, Mar 24). *XGBoost Explained: A Beginner's Guide*. Retrieved from Medium: https://medium.com/low-code-for-advanced-data-science/xgboost-explained-a-beginners-guide-095464ad418f

GeeksforGeeks. (2024, July 15). *K-Nearest Neighbours - GeeksforGeeks*. GeeksforGeeks. https://www.geeksforgeeks.org/k-nearest-neighbours/

IBM. (2021, October 4). *KNN*. Ibm.com. https://www.ibm.com/think/topics/knn

Md Shah, J., Hussin, R., & Idris, A. (2023). POVERTY ERADICATION PROJECT IN SABAH, MALAYSIA: NEW INITIATIVE, NEW CHALLENGES? In *Journal of the Malaysian Institute of Planners VOLUME* (Vol. 21).

ML | XGBoost (eXtreme Gradient Boosting). (2023, Dec 06). Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/ml-xgboost-extreme-gradient-boosting/

Mohd Nor Radieah, & Khelghat-Doost Hamoon. (2019). 1AZAM Programme: The Challenges and Prospects of Poverty Eradication in Malaysia. *International Journal of Academic Research in Business and Social Sciences*, *9*(1), 345–356. https://doi.org/10.6007/ijarbss/v9-i1/5420

Rahman, M. A., Sani, N. S., Hamdan, R., Othman, Z. A., & Bakar, A. A. (2021). A clustering approach to identify multidimensional poverty indicators for the bottom 40 percent group. *PLoS ONE*, *16*(8 August). https://doi.org/10.1371/journal.pone.0255312

Peninsular Malaysia's oil and gas production dropped by half over last decade: Economy Minister. (2024, nov 2024). Retrieved from Cna: https://www.channelnewsasia.com/asia/malaysia-oil-gas-production-peninsula-rafizi-ramli-decline-sabah-sarawak-petronas-4754251#:~:text=According%20to%20the%20federal%20government,up%20around%2018.8%20per%20cent.&text=Sarawak%20also%20accounts%20for%20clos

Poverty and Inequality. (n.d.). Retrieved from The World Bank: https://datatopics.worldbank.org/world-development-indicators/themes/poverty-and-inequality.html

What is the k-nearest neighbors (KNN) algorithm? (n.d.). Retrieved from IBM: https://www.ibm.com/think/topics/knn

XGBoost Documentation. (2022). Retrieved from xgboost developers.: https://xgboost.readthedocs.io/en/latest/index.html#xgboost-documentation

APPENDICES

Appendix A: Dataset sources

1. Poverty by Administrative District: https://open.dosm.gov.my/data-catalogue/hh_poverty_district
2. Household Income and Expenditure by Administrative Districts: https://data.gov.my/data-catalogue/hh_income_district
3. Annual Labor Force by Districts: https://open.dosm.gov.my/data-catalogue/lfs_district?state
4. Education completion rate by state : https://data.gov.my/data-catalogue/completion_school_state
5. Education enrollment by district : https://data.gov.my/data-catalogue/enrolment_school_district
6. CPI inflation : https://data.gov.my/data-catalogue/cpi_state_inflation?state=kelantan&division=02&visual=table

Appendix B: Google Drive Link

The link provided links to a google drive that includes our python code, report and datasets.
Link: https://drive.google.com/drive/folders/1wuEC17dTxT_E2vy4u2nordTT60G-mZ2H?usp=drive_link

Appendix C: Proof of publish on Medium